

1. Linear Sieve 2
2. Mobius Function 2
3. Segmented Sieve 2
4. Pollard Rho 2
5. Miller Rabin 2
6. Phi Function 3
7. Discrete Log / Baby Step Giant Step 3
8. Extended Euclid 3
9. Modular Inverse / Congruence 3
10. nCr / nPr / Catalan / Stars and Bars 3
11. Matrix Exponentiation 4
12. Linear Diophantine Equation 4

1. Dijkstra 5
2. Bellman Ford 5
3. 2-SAT 5
4. LCA 6
5. Bridge Tree 6
6. Articulation Point 7
7. SCC / Tarjan 7
8. HLD 7
9. Dinic Max Flow 8
10. Hopcroft Karp 9
11. Prim's MST 9
12. Re-rooting DP 9
13. Centroid Decomposition 10
14. Treap 10
15. Floyd Warshall 22

1. Fenwick Tree 11
2. Segment Tree + Lazy 11
3. Iterative Segment Tree 12
4. Merge Sort Tree 12
5. Sparse Table 12
6. Trie 12
7. Sqrt Decomposition 13
8. Mo's Algorithm 13
9. Ordered Set 14
10. DSU 14
11. Dynamic Bitset 14

1. Hashing Interval 15
2. Hashing with Update 15
3. Z Algorithm 15
4. KMP 15
5. Manacher 15
6. Aho Corasick 15
7. Suffix Array 16

1. Next Greater Left 17

2. Next Greater Right 17
3. Next Lower Left 17
4. Next Lower Right 17
5. Count Inversion 17
6. Majority Candidate in Range 17

1. Xor Basis / Linear Basis 18
2. Bitmask Formula 21

1. 2D Geometry All in One 18
2. Segment / Line Intersection 19
3. Polygon Area / Perimeter 19
4. Convex Hull 20
5. Convex Diameter 20
6. Point in Convex Polygon 20
7. Integer Geometry / Pick's Theorem 20

1. Grundy Number 20

1. Unordered Hashmap 21
2. 128 Bit IO 21
3. PBDS 21
4. Derangement 21
5. Stirling Number 21
6. Egg Dropping DP 22
7. FFT Multiply 22
8. LCM Formula / LCM Counting 24
9. Stress Testing 24
16. Dijkstra 4

```
//Sublime(Linux)
{"cmd" : "g++ -std=c++14 $file_name -o
$file_base_name && timeout 4s
./$file_base_name<inputf.in>outputf.in
2>output.error",
"shell" : true,
"selector" : "source.c",
"working_dir" : "$file_path",
"file_regex" : "^(.[^:]*):([0-9]+)?(?:[0-9]+)?(?:
.*)$",}

***DEBUG***
ofstream err("output.error");
#ifdef DEBUG
    #define db(v)   #v":" << (v)
    #define clog   err
#else
    #define db(...) ""
    #define clog   if(0)cerr
#endif
template<class Con, class =
decltype(begin(declval<Con>()))>
typename enable_if<!is_same<Con,
string>::value, ostream&>::type
operator<<(ostream& out, const
Con& v) {
    out << "";
    for(auto &i : v){
        out << " " << i;
    }
    return out << "\n";
}
```

***Linear Sieve**

```

int hp[N]; bool isPrime[N];
vector<int> primes, mu(N);
void sieve(){
    mu[1] = 1;
    for(int i = 2; i < N; i++){
        if(!hp[i]){
            hp[i] = i; isPrime[i] = 1;
            primes.push_back(i);
            mu[i] = -1;
        }
        for(auto &p : primes){
            if(p > hp[i] or i * p >=
N)break;
            hp[i * p] = p;
            if(i%p == 0){
                mu[i*p] = 0;
                break;
            }
            else mu[i*p] = -mu[i];
        }
    }
    for(int i = 1; i < N; i++){
        for(int j = i; j < N; j += i){
            divs[j].push_back(i);
        }
    }
}

```

Mobius function

Core :
 $\gcd(a, b) == 1 \Rightarrow \sum \mu[d] \text{ where } d \mid \gcd(a, b)$

$\text{cnt}[d]$ = count of elements in array
divisible by d

1) Ordered coprime pairs in range $[1..n]$
 $\text{ans} = \sum_{d=1}^n \mu[d] * (n / d)^2$

2) Array coprime pairs
 $\text{ans} = \sum_{d=1}^{\max A} \mu[d] * nCr(\text{cnt}[d], 2)$

3) Array pairs with $\gcd = g$
 $\text{ans} = \sum_{d=1}^{\max A/g} \mu[d] * nCr(\text{cnt}[g * d], 2)$

4) Array coprime triples
 $\text{ans} = \sum_{d=1}^{\max A} \mu[d] * nCr(\text{cnt}[d], 3)$

5) Array triples with $\gcd = g$
 $\text{ans} = \sum_{d=1}^{\max A/g} \mu[d] * nCr(\text{cnt}[g * d], 3)$

6) Non-empty subsets with $\gcd = 1$
 $\text{ans} = \sum_{d=1}^{\max A} \mu[d] * (2^{\text{cnt}[d]} - 1)$

7) Non-empty subsets with $\gcd = g$
 $\text{ans} = \sum_{d=1}^{\max A/g} \mu[d] * (2^{\text{cnt}[g * d]} - 1)$

8) Count $x \leq m$ s.t. $\gcd(x, n) = 1$
 $\text{ans} = \sum_{d \mid n} \mu[d] * (m / d)$

9) Euler phi using Mobius
 $\phi(n) = \sum_{d \mid n} \mu[d] * (n / d)$

10) Square-free check
 $\text{is_square_free}(n) = \mu[n] * \mu[n]$

```

int countOrderedCoprimePairs(){
    int ans = 0;
    for(int d = 1; d <= mx; d++){
        ans += mu[d] * cnt[d] *
(cnt[d] - 1) / 2;
    }
    return ans;
}

```

*** Segmented Sieve ***

```

vector<char> segmentedSieve(int
L, int R) {
    int lim = sqrt(R);
    vector<char> mark(lim + 1,
false);
    vector<int> primes;
    for (int i = 2; i <= lim; ++i)
    {
        if (!mark[i]) {
            primes.emplace_back(i);
            for (int j = i * i; j <=
lim; j += i)
                mark[j] = true;
        }
    }
    vector<char> isPrime(R - L +
1, true);
    for (int i : primes)
        for (int j = max(i * i, (L +
i - 1) / i * i); j <= R; j += i)
            isPrime[j - L] = false;
    if (L == 1)
        isPrime[0] = false;
    return isPrime;
}
/*
vector<char> res =
segmentedSieveNoPreGen(1, r);

```

```

for(int i=0 ; i<res.size() ;
i++) if(a[i]) cout << (1 + i)
<< ' '; cout << endl;
TC: n loglog(R) +
sqrt(R)loglogsqrt(R); n = R - L
+ 1;
*/
***PollardRho***
namespace Pollard{
    #define int long long
    mt19937
rnd(chrono::steady_clock::now().
time_since_epoch().count());
    const int N = 1e6 + 100;
    bool isPrime[N]; int hp[N];
    vector<int> primes;
    void init(){
        for(int i = 2; i < N; i++){
            if(!hp[i]){
                hp[i] = i; isPrime[i] = 1;
                primes.push_back(i);
            }
            for(auto &p : primes){
                if(p > hp[i] or i * p >=
N)break;
                hp[i * p] = p;
            }
        }
        int binExp(int a, int b, int
mod = M){
            int ans = 1; a %= mod;
            for(; b; b >>= 1, a =
((__uint128_t) a * a % mod){
                if(b & 1)(ans =
((__uint128_t) ans * a % mod);
            }
            return ans;
        }
        int mmiOfX(int b, int mod =
M){ return binExp(b, mod - 2);}
        bool is_composite(int n, int
a, int d, int p){
            int x = binExp(a, d, n);
            if(x == 1 or x == n - 1)return
false;
            for(int i = 0; i < p - 1;
i++){
                x = ((__uint128_t)x * x % n;
                if(x == n - 1)return false;
            }
            return true;
        }
    }
    **MILLER_RABIN**
    bool is_prime(int n, int iter

```

```

= 5){
    if(n < 4)return n == 2 || n ==
3;
    int p = 0, d = n - 1;
    while((d & 1) == 0){
        d >>= 1;
        p++;
    }
    mt19937_64 rn;
    for(int i = 0; i < iter; i++){
        int a = 2 + rn() % (n - 3);
        if(is_composite(n, a, d,
p))return false;
    }
    return true;
}
//Pollard rho
int func(int x, int c, int
mod) {
    return ((__uint128_t)x * x +
c) % mod;
}
int rho(int n) {
    if (n % 2 == 0) return 2;
    while (true) {
        int c = 1 + rnd() % (n - 1);
        int x = 2 + rnd() % (n - 2);
        int y = x;
        int d = 1;
        while (d == 1) {
            x = func(x, c, n);
            y = func(func(y, c, n), c,
n);

            int diff = abs(x - y);
            d = __gcd(diff, n);
        }
        if (d != n) return d;
    }
}
vector<int> factorize(int n){
    if(n == 1)return {};
    if(is_prime(n))return {n};
    vector<int> v, w;
    if(n < N){
        while(n > 1){
            int pm = hp[n];
            while(n % pm == 0){
                v.push_back(pm);
                n /= pm;
            }
        }
        return v;
    }
    int x = rho(n);
    v = factorize(x);

```

```

    w = factorize(n / x);
    v.insert(v.end(), w.begin(),
w.end());
    return v;
}
void allDivsGen(int ind, int
curVal, vector<pair<int, int>>
&pmFact, vector<int> &ans){
    if(pmFact[ind].first == -1){
        ans.push_back(curVal);
        return;
    }
    for(int i = 0; i <=
pmFact[ind].second; i++){
        allDivsGen(ind + 1, curVal,
pmFact, ans);
        curVal *= pmFact[ind].first;
    }
}
map<int, vin> dp;
vector<int> allDivisors(int
n){
    auto it = dp.find(n);
    if(it != dp.end())return
it->second;
    vector<int> ans;
    vector<pair<int, int>>
prime_factors;
    vector<int> fact =
factorize(n);
    unordered_map<int, int> mp;
    for(auto i : fact)mp[i]++;
    for(auto i :
mp)prime_factors.push_back({i.fi
rst, i.second});
    prime_factors.push_back({-1,
-1});
    allDivsGen(0, 1,
prime_factors, ans);
    int x = ans.size();
    return dp[n] = ans;
}
}

*** Phi function ***
int phi(int n) {
    int result = n;
    for (int i = 2; i * i <= n;
i++) {
        if (n % i == 0) {
            while (n % i == 0)
                n /= i;
            result -= result / i;
        }
    }
}

```

```

    if (n > 1)
        result -= result / n;
    return result;
}

***Returns minimum x for
which  $a^x \% m = b \% m$ , a
and m are not coprime.
int solve(int a, int b, int m) {
    a %= m, b %= m;
    int k = 1, add = 0, g;
    while ((g = __gcd(a, m)) > 1)
    {
        if (b == k)
            return add;
        if (b % g)
            return -1;
        b /= g, m /= g, ++add;
        k = (k * 111 * a / g) % m;
    }

    int n = sqrt(m) + 1;
    int an = 1;
    for (int i = 0; i < n; ++i)
        an = (an * 111 * a) % m;

    unordered_map<int, int> vals;
    for (int q = 0, cur = b; q <=
n; ++q) {
        vals[cur] = q;
        cur = (cur * 111 * a) % m;
    }

    for (int p = 1, cur = k; p <=
n; ++p) {
        cur = (cur * 111 * an) % m;
        if (vals.count(cur)) {
            int ans = n * p -
vals[cur] + add;
            return ans;
        }
    }
    return -1;
}

***Returns minimum x for
which  $a^x \% m = b \% m$ , a
and m are coprime. ***
int solve(int a, int b, int m) {
    a %= m, b %= m;
    int n = sqrt(m) + 1;
    int an = 1;
    for (int i = 0; i < n; ++i)
        an = (an * 111 * a) % m;

    unordered_map<int, int> vals;
    for (int q = 0, cur = b; q <=

```

```

n; ++q) {
    vals[cur] = q;
    cur = (cur * 111 * a) % m;
}
for (int p = 1, cur = 1; p <=
n; ++p) {
    cur = (cur * 111 * an) % m;
    if (vals.count(cur)) {
        int ans = n * p -
vals[cur];
        return ans;
    }
}
return -1;
}

*** Extended euclid ***
int G;
pair<int, int> e_gcd(int a, int
b){
    //ax + by = gcd(a, b); x, y = ?
    if(!b){
        G = a;
        return {1, 0};
    }
    auto [x1, y1] = e_gcd(b, a%b);
    int x = y1;
    int y = x1 - (a / b)*y1;
    return {x, y};
}

int mmiofX(int a, int mod){
    return (mod + e_gcd(a,
mod).first)%mod;
}

int congruence(int a, int b, int
m){ //ax = b mod(m); x = ?
    int g = gcd(a, m);
    if(b % g)return 0;
    a /= g, b /= g, m /= g;
    int a1 = mmiofX(a, m);
    int x = (b * a1)%m;
    return x;
}

*** nCr ***
int binExp(int a, int b, int mod
= M){
    int ans = 1; a %= mod;
    for(; b; b >>= 1, a =
(__uint128_t) a * a % mod){
        if(b & 1)(ans =
(__uint128_t) ans * a % mod);
    }
    return ans;
}

```

```

int bigMult(int a, int b, int
mod = M){
    int ans = 0;
    for(; b; b >>= 1, a += a){
        if(b&1)ans += a;
    }
    return ans;
}

int mmiofX(int b, int mod = M){
    return binExp(b, mod - 2);}

int fact[N], inv[N];
void init(){
    fact[0] = inv[0] = 1;
    for(int i = 1; i < N; i++){
        fact[i] = fact[i - 1] * i %
M;
    }
    inv[N - 1] = mmiofX(fact[N -
1]);
    for(int i = N - 2; i >= 0;
i--){
        inv[i] = inv[i + 1] * (i +
1) % M;
    }
    /*
    for(int n = 1; n <= 66;
n++){
        nCr[n][0] = nCr[n][n] = 1;
        for(int r = 1; r < n;
r++){
            nCr[n][r] = nCr[n - 1][r
- 1] + nCr[n - 1][r];
        }
    }
    */
}

int nCr(int n, int r){return
(__uint128_t) (fact[n] * inv[r]
% M) * inv[n - r] % M; }
int nPr(int n, int r){return
(__uint128_t) fact[n] * inv[n -
r] % M; }
int catalan(int n){return
nCr(2*n, n) * mmiofX(n + 1) % M;
}

int starsAndBars(int n, int
k){return nCr(n - 1 + k, k);}

***Matrix exponentiation***
#define v2d vector<vector<int>>
v2d fib_base = {{1, 1}, {1, 0}};
//for Fibonacci number
v2d matrixMult(v2d &a, v2d &b){
    int m = a.size(), n =

```

```

a[0].size(), l = b[0].size();
v2d ans(m, vector<int> (1));
for(int i = 0; i < m; i++){
    for(int j = 0; j < l; j++){
        for(int k = 0; k < n; k++){
            ans[i][j] += a[i][k] *
b[k][j] % M;
            ans[i][j] %= M;
        }
    }
    return ans;
}

v2d matrixExpo(v2d v, int p){
    v2d ans = v;
    for(; p; p >>= 1, v =
matrixMult(v, v)){
        if(p&1)ans = matrixMult(ans,
v);
    }
    return ans;
}

*** Diophantine
int gcd(int a, int b, int& x,
int& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int d = gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

bool find_any_solution(int a,
int b, int c, int &x0, int &y0,
int &g) {
    if(a == 0 and b == 0) {
        if(c) return false;
        x0 = y0 = g = 0;
        return true;
    }
    g = gcd(abs(a), abs(b), x0,
y0);
    if (c % g) return false;
    x0 *= c / g;
    y0 *= c / g;
    if (a < 0) x0 = -x0;
    if (b < 0) y0 = -y0;
    return true;
}

void shift_solution(int & x, int
& y, int a, int b, int cnt) {

```

```

    x += cnt * b;
    y -= cnt * a;
}
int find_all_solutions(int a,
int b, int c, int minx, int
maxx, int miny, int maxy) {
    int x, y, g;
    if (!find_any_solution(a, b,
c, x, y, g)) return 0;
    if (a == 0 and b == 0) {
        assert(c == 0);
        return 1LL * (maxx - minx +
1) * (maxy - miny + 1);
    }
    if (a == 0) {
        return (maxx - minx + 1) *
(miny <= c / b and c / b <=
maxy);
    }
    if (b == 0) {
        return (maxy - miny + 1) *
(minx <= c / a and c / a <=
maxx);
    }
    a /= g;
    b /= g;
    int sign_a = a > 0 ? +1 : -1;
    int sign_b = b > 0 ? +1 : -1;
    shift_solution(x, y, a, b,
(minx - x) / b);
    if (x < minx)
        shift_solution(x, y, a, b,
sign_b);
    if (x > maxx)
        return 0;
    int lx1 = x;
    shift_solution(x, y, a, b,
(maxx - x) / b);
    if (x > maxx)
        shift_solution(x, y, a, b,
-sign_b);
    int rx1 = x;
    shift_solution(x, y, a, b,
-(miny - y) / a);
    if (y < miny)
        shift_solution(x, y, a, b,
-sign_a);
    if (y > maxy)
        return 0;
    int lx2 = x;
    shift_solution(x, y, a, b,
-(maxy - y) / a);
    if (y > maxy)
        shift_solution(x, y, a, b,
sign_a);
    int rx2 = x;

```

```

    if (lx2 > rx2)
        swap(lx2, rx2);
    int lx = max(lx1, lx2);
    int rx = min(rx1, rx2);
    if (lx > rx)
        return 0;
    return (rx - lx) / abs(b) + 1;
}
// solution exist or not?
// how many solution exist?

*** Dijkstra ***
void dijkstra(){
    int n, m;
    cin >> n >> m;
    vector<pair<int, int>> graph[n
+ 1];
    vector<int> dist(n + 1, 1e18);

    for(int i = 0; i < m; i++){
        int x, y, w;
        cin >> x >> y >> w;
        graph[x].push_back({y, w});
        graph[y].push_back({x, w});
    }

    int u = 1; // source node
    dist[u] = 0;
    set<pair<int, int>> s;
    s.insert({0, u});

    while(!s.empty()){
        auto [w, u] = *s.begin();
        s.erase(s.begin());
        if(dist[u] < w)continue;

        for(auto &[v, ww] : graph[u]){
            if((w + ww) < dist[v]){
                dist[v] = w + ww;
                s.insert({dist[v], v});
            }
        }
    }
}

*** Bellman ford ***
vector<pair<ll, ll>> g[N];
vector<ll> dist(N), par(N);
ll n, m;
void bellman_ford(ll src) {
    for(ll i = 0; i <= n; i++)
dist[i] = INF;
    dist[src] = 0;
    for(ll step = 0; step < n - 1;
step++) {

```

```

        //1 based indexing
        for(ll i = 1; i <= n; i++) {
            //it.ff = node, it.ss =
weight
            for(auto it : g[i]) {
                if(dist[i] + it.ss <
dist[it.ff]) {
                    par[it.ff] = i;
                    dist[it.ff] = dist[i]
+ it.ss;
                }
            }
        }
    }
}

void cycle_detect() {
    ll cycle = 0;
    for(ll i = 1; i <= n; i++) {
        for(auto it : g[i]) {
            if(dist[i] + it.ss <
dist[it.ff]) {
                cycle = it.ff;
                break;
            }
        }
    }
    if(cycle == 0) {
        cout << "NO" << endl;
    } else {
        cout << "YES" << endl;
        for(ll i = 0; i < n; i++)
cycle = par[cycle];
        vector<ll> ans;
        ans.pb(cycle);
        for(ll i = par[cycle]; i !=
cycle; i = par[i]) {
            ans.pb(i);
        }
        ans.pb(cycle);
        reverse(all(ans));
        for(auto it : ans) cout <<
it << " ";
        cout << endl;
    }
}

*** 2-sat ***
vector<ll> g[N], g2[N];
vector<ll> order, vis(N),
comp(N);
vector<char> ans(N);
ll n, m;
void dfs(ll node) {
    vis[node] = 1;
    for(auto child : g[node]) {
        if(!vis[child]) dfs(child);
    }
}

```

```

    order.pb(node);
}
void dfs2(ll node, ll cnt) {
    comp[node] = cnt;
    for(auto child : g2[node]) {
        if(!comp[child]) dfs2(child,
cnt);
    }
}
void createComponentGraph() {
    order.clear();
    loop(i, 1, n) vis[i] = 0;
    for(int node = 1; node <= 2 *
m; node++) {
        if(!vis[node]) dfs(node);
    }
    reverse(all(order));
    ll cnt = 0;
    for(auto it : order) {
        if(!comp[it]) {
            dfs2(it, ++cnt);
        }
    }
    for(int node = 1; node <= m;
node++) {
        if(comp[node] == comp[2 * m
- node + 1]) {
            cout << "IMPOSSIBLE" <<
endl;
            return;
        }
        // if x is in left of !x in
topological order then x = 0,
else x = 1
        ans[node] = (comp[node] >
comp[2 * m - node + 1] ? '+' :
'-');
    }
    for(ll i = 1; i <= m; i++) {
        cout << ans[i];
    }
    cout << endl;
}
void SAT2() {
    cin >> n >> m;
    for(ll i = 0; i < n; i++) {
        char ch1, ch2;
        ll u, v;
        cin >> ch1 >> u >> ch2 >> v;
        if(ch1 == '-') u = 2 * m - u
+ 1;
        if(ch2 == '-') v = 2 * m - v
+ 1;
        // u = x, 2 * m - u + 1 = !x
        g[2 * m - u +
1].push_back(v);

```

```

        g[2 * m - v +
1].push_back(u);
        g2[u].push_back(2 * m - v +
1);
        g2[v].push_back(2 * m - u +
1);
    }
    createComponentGraph();
}
***LCA***
const int N = 2e5 + 5, LG = 20;
vector<int> g[N];
int parent[N][LG + 1], dep[N];
// int mx[N][LG + 1], mn[N][LG +
1];
void dfs(int u, int p) { // take
parameter mxi, mni for max min
    parent[u][0] = p;
    dep[u] = dep[p] + 1;
    // mx[u][0] = mxi;    mn[u][0]
= mni;
    for (int i = 1; i <= LG; i++)
    {
        parent[u][i] =
parent[parent[u][i - 1]][i - 1];
        // mx[u][i] = max(mx[u][i -
1], mx[ parent[u][i - 1] ][i -
1]);
        // mn[u][i] = min(mn[u][i -
1], mn[ parent[u][i - 1] ][i -
1]);
    }
    for (auto v: g[u]) if (v != p)
    {
        dfs(v, u);
    }
}
int LCA(int u, int v) {
    if (dep[u] < dep[v])
        swap(u, v);
    for (int k = LG; k >= 0; k--)
        if (dep[parent[u][k]] >=
dep[v])
            u = parent[u][k];
    if (u == v) return u;
    for (int k = LG; k >= 0; k--)
        if (parent[u][k] !=
parent[v][k])
            u = parent[u][k], v =
parent[v][k];
    return parent[u][0];
}
int kth(int u, int k) {
    assert(k >= 0);
    for (int i = 0; i <= LG; i++)

```

```

        if (k & (1 << i))
            u = parent[u][i];
        return u;
    }
    int dist(int u, int v) {
        int l = LCA(u, v);
        return dep[u] + dep[v] -
(dep[l] << 1);
    }
    // Kth from [u -> v], 0th node
is u
    int go(int u, int v, int k) {
        int l = LCA(u, v);
        int d = dep[u] + dep[v] -
(dep[l] << 1);
        assert(k <= d);
        if (dep[l] + k <= dep[u])
            return kth(u, k);
        k -= dep[u] - dep[l];
        return kth(v, dep[v] - dep[l]
- k);
    }
    // dep[0] = -1
    // dfs(u, 0)
    // TC -> build: NlogN
    // -> query: logN

    *** Bridge Tree ***
    int n, m;
    vector<int> g[N], t[N];
    map<pair<int, int>, int> eid;
    int timer, in[N], low[N];
    bool is_bridge[N], vis[N];
    void dfs(int u, int p){
        in[u] = low[u] = ++timer;
        vis[u] = true;
        for(auto &v : g[u])if(v != p){
            if(vis[v]){ //Back edge
                low[u] = min(low[u], in[v]);
            }else{ //Forward edge
                dfs(v, u);
                low[u] = min(low[u],
low[v]);
                if(in[u] < low[v]){
                    is_bridge[eid[{u, v}]] =
true; //this edge is a bridge
                } } }
    }
    int comp[N], comp_sz[N], cid;
    void compress(int u, int &cnt){
        comp[u] = cid;
        cnt++;
        for(auto &v : g[u]){
            int id = eid[{u, v}];
            if(!comp[v] and
!is_bridge[id]){
                compress(v, cnt);
            }
        }
    }

```



```

    } } }
void reset(){
    eid.clear();
    timer = cid = 0;
    for(int i = 0; i <= max(n, m);
i++){
    g[i].clear(); t[i].clear();
    comp[i] = comp_sz[i] = 0;
    in[i] = 0, low[i] = vis[i] =
0, is_bridge[i] = 0;
    }}
void init(){
    cin >> n >> m;
    reset();
    vector<pair<int, int>> edg(m +
1);
    for(int i = 1; i <= m; i++){
        int x, y;
        cin >> x >> y;
        g[x].push_back(y);
        g[y].push_back(x);
        edg[i] = {x, y};
        eid[{x, y}] = eid[{y, x}] = i;
    }
    dfs(1, 0); //If graph contain
only one CC
    for(int i = 1; i <= n; i++){
        if(!comp[i]){
            ++cid;
            compress(i, comp_sz[cid]);
        }
    }
    for(int i = 1; i <= m; i++){
        if(!is_bridge[i])continue;
        int u = edg[i].first;
        int v = edg[i].second;
        u = comp[u]; v = comp[v];
        t[u].push_back(v);
        t[v].push_back(u);
    }
}

```

*** Articulation point ***

```

int timer;
set<int> art_point;
vector<int> in(N), low(N),
vis(N);

void dfs(int u, int p = 0){
    vis[u] = true;
    in[u] = low[u] = timer++;
    int cnt = 0;

    for(auto &v : graph[u])if(v !=
p){
        if(vis[v]){
            low[u] = min(low[u], in[v]);
        }else{

```

```

            cnt++;
            dfs(v, u);
            if(low[v] >= in[u] and p){
                art_point.insert(u);
            }
            low[u] = min(low[u],
low[v]);
        }
    }
    if(!p and cnt > 1){
        art_point.insert(u);
    }
}
void find_cutpoints(){
    timer = 0;
    in.assign(n + 1, 0);
    low.assign(n + 1, 0);
    vis.assign(n + 1, 0);
    art_point.clear();

    for(int i = 1; i <= n; i++){
        if(!vis[i])dfs(i);
    }
}
***SCC**
int n;
int timer = 0;
stack<int> st;
vector<vector<int>> scc,
graph(N);
vector<int> in(N, -1), low(N,
-1), inStack(N);

void dfs(int u){
    in[u] = low[u] = timer++;
    st.push(u);
    inStack[u] = true;
    for(auto &v : graph[u]){
        if(in[v] == -1){
            dfs(v);
            low[u] = min(low[u],
low[v]);
        }else if(inStack[v]){
            low[u] = min(low[u], in[v]);
        }
    }
    if(low[u] == in[u]){
        vector<int> cc;
        while(true){
            int v = st.top(); st.pop();
            inStack[v] = false;
            cc.push_back(v);
            if(v == u)break;
        }
        scc.push_back(cc);
    }
}

```

```

}
//make new graph considering
each scc as a node
vector<pii> re_edges(vector<pii>
&edges){
    vector<int> cid(n + 1);
    int m = 0;
    for(auto &v : scc){ ++m;
    for(auto i : v)cid[i] = m;
    }
    vector<pii> ans;
    for(auto &[u, v] : edges){
        if(cid[u] == cid[v])continue;
        ans.push_back({cid[u],
cid[v]});
    }
    sort(ans.begin(), ans.end());
    ans.erase(unique(ans.begin(),
ans.end()), ans.end());
    return ans;
};
*** HLD ***
//Graph + LCA part
int n, m, q;
const int LG = 19;
vector<int> graph[N];
int par[N][LG + 1], sz[N],
dep[N];

void init(int u, int p){
    sz[u] = 1;
    par[u][0] = p;
    for(int i = 1; i <= LG;
i++)par[u][i] = par[par[u][i -
1]][i - 1];

    if(p)graph[u].erase(find(all gra
ph[u]), p));
    for(auto &v : graph[u])if(v !=
p){
        dep[v] = dep[u] + 1;
        init(v, u);
        sz[u] += sz[v];
        auto &x = graph[u][0];
        if(sz[v] > sz[x])swap(v, x);
    }
}
int kth(int u, int k){
    if(k < 0)k = 0;
    while(k){
        int lg = 31 -
__builtin_clz(k); //log2(k)
        k -= (1 << lg);
        u = par[u][lg];
    }
    return u;
}

```

```

}
int lca(int u, int v){
    if(dep[u] < dep[v])swap(u, v);
    u = kth(u, dep[u] - dep[v]);
    if(u == v)return u;
    for(int i = LG; i >= 0; i--){
        if(par[u][i] ==
par[v][i])continue;
        u = par[u][i];
        v = par[v][i];
    }
    return par[u][0];
}
//HLD part
int cur_pos = 0;
vector<int> head(N), pos(N);

void hld(int u = 1){
    pos[u] = ++cur_pos;
    for(auto &v : graph[u]){
        if(v == graph[u][0])head[v] =
head[u];
        else head[v] = v;
        hld(v);
    }
}
//Segment Tree
vector<int> arr, t(4 * N + 1);
const int R = 0;
inline int pull(int a, int b){
    return a + b;
}
#define lc node << 1
#define rc node << 1 | 1
void build(int node, int lo, int
hi){
    if(lo == hi){
        t[node] = arr[lo];
        return;
    }
    int mid = (hi + lo) >> 1;
    build(lc, lo, mid);
    build(rc, mid + 1, hi);
    t[node] = pull(t[lc], t[rc]);
}
void update(int node, int lo,
int hi, int l, int x){
    if(hi < l or lo > l)return;
    if(lo == hi){
        t[node] = arr[lo] = x;
        return;
    }
    int mid = (hi + lo) >> 1;
    update(lc, lo, mid, l, x);
    update(rc, mid + 1, hi, l, x);
    t[node] = pull(t[lc], t[rc]);
}

```

```

}
int query(int node, int lo, int
hi, int l, int r){
    if(hi < l or lo > r)return R;
    if(lo >= l and hi <= r)return
t[node];
    int mid = (hi + lo) >> 1;
    auto q1 = query(lc, lo, mid,
l, r);
    auto q2 = query(rc, mid + 1,
hi, l, r);
    return pull(q1, q2);
}
void Update(int l, int x){
    update(1, 1, n, l, x);
}
int Query(int l, int r){
    return query(1, 1, n, l, r);
}
//HLD query
int query_up(int u, int v){
    int ans = 0;
    while(head[u] != head[v]){
        int chain_head = head[u];
        int cur =
Query(pos[chain_head], pos[u]);
        u = par[chain_head][0];
        ans = pull(ans, cur);
    }
    int cur = Query(pos[v],
pos[u]);
    ans = pull(ans, cur);
    return ans;
}
int query(int u, int v){
    int l = lca(u, v);
    int ans = pull(query_up(u, l),
query_up(v, l)) - query_up(l,
l); //(-) for sum query
    return ans;
}
void reset(){
    cur_pos = 0;
    arr.assign(n + 1, 0);
    for(int i = 0; i <= n; i++){
        graph[i].clear();
        pos[i] = head[i] = 0;
        dep[i] = sz[i] = 0;
    }
}
void solve(){
    cin >> n;
    int vals[n + 1];
    for(int i = 1; i <= n; i++)cin
>> vals[i];
    for(int i = 1; i < n; i++){

```

```

int x, y;
cin >> x >> y;
graph[x].push_back(y);
graph[y].push_back(x);
}
init(1, 0);
head[1] = 1;
hld(1);
for(int i = 1; i <= n;
i++)arr[pos[i]] = vals[i];
build(1, 1, n);

//enjoy here
}
*** MaxFlow(Dinic) ***
struct Dinic{
    struct edge{
        int u, v;
        int cap = 0, flow = 0;
        edge(int u = 0, int v = 0, int
cap = 0) : u(u), v(v),
cap(cap){}

        int other(int x){return x == u
? v : u;}
        int flowFrom(int x){return x
== u ? flow : -flow;}
        int rescap(int x){return x ==
u ? cap - flow : flow;}
        void sendFlow(int x, int
f){flow += (x == u) ? f : -f;}
    };
    int n = 0, clk = 0;
    vector<edge> E;
    vector<vector<int>> adj;
    vector<int> vis, level, ptr;
    Dinic(int n) : n(n), adj(n +
1), vis(n + 1){
        level = ptr = vis;
    };
    inline void addEdge(int u, int
v, int cap){
        E.emplace_back(u, v, cap);
        adj[u].push_back(E.size() -
1);
        adj[v].push_back(E.size() -
1);
    }
    bool bfs(int s, int t){
        clk++;
        queue<int> q;
        q.push(s);
        level[s] = 0;
        vis[s] = clk;

        while(!q.empty()){

```



```

    s = q.front(); q.pop();
    if(s == t)break;
    ptr[s] = 0;
    for(auto i : adj[s]){
        int v = E[i].other(s);
        if(vis[v] == clk or
E[i].rescap(s) == 0)continue;
        vis[v] = clk;
        level[v] = level[s] + 1;
        q.push(v);
    }
    return vis[t] == clk;
}
int augment(int u, int flow,
int t){
    if(u == t or !flow)return
flow;
    for(int &i = ptr[u]; i <
adj[u].size(); i++){
        edge &e = E[adj[u][i]];
        int v = e.other(u);
        if(vis[v] != clk)continue;
        if(e.rescap(u) ==
0)continue;
        if(level[v] != (level[u] +
1))continue;
        int sent = augment(v,
min(flow, e.rescap(u)), t);
        if(sent){
            e.sendFlow(u, sent);
            return sent;
        }
    }
    level[u] = -1;
    return 0;
}
int maxFlow(int s, int t){
    int res = 0;
    while(bfs(s, t)){
        while(int temp = augment(s,
1e18, t)){
            res += temp;
        }
    }
    return res;
}
};
**Hopcroft Karp(Max bipartite
match)**
struct HopcroftKarp {
    int n, m;
    vector<vector<int>> g;
    vector<int> matchL, matchR,
dist;

```

```

    HopcroftKarp(int n, int m) :
n(n), m(m), matchL(n+1,0),
matchR(m+1,0), dist(n+1), g(n+1)
{}
    void add_edge(int u, int v) {
        g[u].push_back(v); }
    bool bfs() {
        queue<int> q; bool found =
false;
        for(int i=1;i<=n;i++) {
            if(!matchL[i]) { dist[i]=0;
q.push(i); }
            else dist[i]=INF;
        }
        while(!q.empty()) {
            int u = q.front(); q.pop();
            for(int v : g[u]) {
                if(matchR[v] &&
dist[matchR[v]]==INF) {
                    dist[matchR[v]] =
dist[u]+1;
                    q.push(matchR[v]);
                }
                if(!matchR[v]) found=true;
            } }
        return found;
    }
    bool dfs(int u) {
        for(int v : g[u]) {
            if(!matchR[v] ||
(dist[matchR[v]]==dist[u]+1 &&
dfs(matchR[v]))) {
                matchL[u]=v; matchR[v]=u;
                return true;
            } }
        dist[u]=INF;
        return false;
    }
    int maxMatching() {
        int ans=0;
        while(bfs())
            for(int i=1;i<=n;i++)
                if(!matchL[i]) ans+=dfs(i);
        return ans;
    }
};
*** Prims ***
void prims(){
    int n, m;
    cin >> n >> m;
    vector<pair<int, int>> graph[n
+ 1];
    for(int i = 0; i < m; i++){
        int x, y, w;
        cin >> x >> y >> w;
        graph[x].push_back({y, w});
    }

```

```

    graph[y].push_back({x, w});
}
    set<pair<int, int>> s;
    s.insert({0, 1});
    vector<bool> vis(n + 1);

    int ans = 0;
    while(!s.empty()){
        auto [w, u] = *s.begin();
        s.erase(s.begin());
        if(vis[u])continue;
        vis[u] = true;
        ans += w;

        for(auto &[v, w] : graph[u]){
            if(!vis[v])s.insert({w, v});
        }
    }
    cout << ans << endl;
}

*** Re-rooting DP***
// rerooting for tree
// create graph
// here...
// calculate for node 1(some fix
node) as a root
void dfs(ll node, ll par) {
    for(auto child : g[node]) {
        if(child.ff != par) {
            // call dfs from child
            dfs(child.ff, node);
            // add contribution of
child to the node
            // here...
        } } }
void dfs2(ll node, ll par) {
    // calculate contributon of
node as a root, ans add to ans
    // here...
    for(auto child : g[node]) {
        if(child.ff != par) {
            // remove contribution
from node first, then add
contribution to child
            // here...
            // call dfs2 from child
            dfs2(child.ff, node);
            // remove contribution
from child first, then add
contribution to node
            // here...
        } } }
*** centroid decomposition **
#define int long long

```

```

vector<vector<int>> up(N,
vector<int>(L));
vector<int> g[N];
vector<int> dep(N);
void dfs(int u, int p) {
    up[u][0] = p;
    dep[u] = dep[p] + 1;
    for(int i = 1; i < L; i++) {
        up[u][i] = up[up[u][i -
1]][i - 1];
    }
    for(auto it : g[u]) {
        if(it != p) dfs(it, u);
    }
}
int lift(int u, int k) {
    int i = 0;
    while(k) {
        if(k & 1) u = up[u][i];
        i++;
        k >>= 1;
    }
    return u;
}
int lca(int u, int v) {
    if(dep[u] < dep[v]) swap(u,
v);
    u = lift(u, dep[u] - dep[v]);
    if(u == v) return u;
    for(int i = L - 1; i >= 0;
i--) {
        if(up[u][i] != up[v][i]) {
            u = up[u][i];
            v = up[v][i];
        }
    }
    return up[u][0];
}
int dist(int u, int v) {
    int res = dep[u] + dep[v] - 2 *
dep[lca(u, v)];
    return res;
}
set<int> g2[N];
int sz[N], pa[N];
int dfs2(int u, int p) {
    sz[u] = 1;
    for(auto v : g2[u]) {
        if(v != p) {
            sz[u] += dfs2(v, u);
        }
    }
    return sz[u];
}
int centroid(int u, int p, int
n) {
    for(auto v : g2[u]) {

```

```

        if(v != p) {
            if(sz[v] > n / 2) return
centroid(v, u, n);
        }
    }
    return u;
}
void build(int u, int p) {
    int n = dfs2(u, p);
    int c = centroid(u, p, n);
    if(p == -1) p = c;
    pa[c] = p;
    vector<int> tmp(g2[c].begin(),
g2[c].end());
    for(auto v : tmp) {
        g2[c].erase(v);
        g2[v].erase(c);
        build(v, c);
    }
}
vector<int> ans(N);
void update(int u) {
    int cur = u;
    while(1) {
        ans[cur] = min(ans[cur],
dist(u, cur));
        if(pa[cur] == cur) break;
        cur = pa[cur];
    }
}
int query(int u) {
    int res = INF;
    int cur = u;
    while(1) {
        res = min(res, ans[cur] +
dist(u, cur));
        if(pa[cur] == cur) break;
        cur = pa[cur];
    }
    return res;
}
void solve() {
    int n, m;
    cin >> n >> m;
    //clear()
    for(int i = 0; i <= n; i++) {
        g[i].clear();
        g2[i].clear();
        ans[i] = INF;
    }
    for(int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        g[u].pb(v);
        g[v].pb(u);
        g2[u].insert(v);

```

```

        g2[v].insert(u);
    }
    dfs(1, 0);
    build(1, -1);
    update(1);
    while(m--) {
        int id, x;
        cin >> id >> x;
        if(id == 1) {
            update(x);
        } else {
            cout << query(x) << endl;
        }
    }
    cout << endl;
}
signed main () {
    #ifndef FARHAN
        freopen("input.txt", "r",
stdin);
        freopen("output.txt", "w",
stdout);
    #endif
    FIO;
    // TC(t)
    solve();
    return 0;
}

*** Treap ***
mt19937
rnd(chrono::steady_clock::now().
time_since_epoch().count());
struct node{
    int key, prior, cnt;
    node *l, *r;
    node(int id){
        l = r = nullptr;
        key = id;
        prior = rnd();
        cnt = 1;
    }
};
struct treap{
    node *root;
    treap() {
        root = nullptr;
    }
    void split(node *t, int pos,
node *&l, node *&r){
        if (!t) {
            l = r = nullptr;
            return;
        }
        if (t->key <= pos) {

```

```

    if (t->r) t->cnt -=
t->r->cnt;
    split(t->r, pos, l, r);
    if (l) t->cnt += l->cnt;
    t->r = l; l = t;
} else {
    if (t->l) t->cnt -=
t->l->cnt;
    split(t->l, pos, l, r);
    if (r) t->cnt += r->cnt;
    t->l = r; r = t;
}
}
node* merge(node *l, node *r){
    if ((!l) || (!r)) return l ?
l : r;
    if (l->prior < r->prior) {
        if(l->r) l->cnt -=
l->r->cnt;
        l->r = merge(l->r, r);
        if (l->r) l->cnt +=
l->r->cnt;
        return l;
    }
    if (r->l) r->cnt -=
r->l->cnt;
    r->l = merge(l, r->l);
    r->cnt += r->l->cnt;
    return r;
}
void insert(int id){
    node *l, *r;
    split(root, id, l, r);
    root = merge(merge(l, new
node(id)), r);
}
node* merge_treap(node *l,
node *r) {
    if ((!l) || (!r)) return l ?
l : r;
    if(l->prior < r->prior)
swap(l, r);
    node *L, *R;
    split(r, l->key, L, R);
    if (l->r) l->cnt -=
l->r->cnt;
    if (l->l) l->cnt -=
l->l->cnt;
    l->r = merge_treap(l->r, R);
    l->l = merge_treap(L, l->l);
    if (l->r) l->cnt +=
l->r->cnt;
    if (l->l) l->cnt +=
l->l->cnt;
    return l;
}

```

```

void combine(node *x) {
//combine with another treap
    root = merge_treap(root, x);
    // int id = x->key;
    // node *l, *r;
    // split(root, id, l, r);
    // root = merge(merge(l, x),
r);
}
node* erase(int L, int R) {
    node *l, *r, *mid, *mr;
    split(root, L - 1, l, r);
    split(r, R, mid, mr);
    root = merge(l, mr);
    mid->l = mid->r = nullptr;
    return mid;
}
int dfs(node *t, int val){
    if (!t) return 0;
    if (t->key >= val)
        return dfs(t->l, val);
    int r = 1;
    if (t->l) r += t->l->cnt;
    r += dfs(t->r, val);
    return r;
}
int query(int val){
    return dfs(root, val);
}
}
*** Fenwick tree ***
template<typename T = int>
struct BIT{
    int n;
    vector<T> t;
    BIT(int n = 2e5) : n(n){
        t.resize(n + 2);
    }

    T query(int i){
        if(i < 0)return 0;
        int ans = 0;
        for(; i; i -= i & -i)ans +=
t[i];
        return ans;
    }

    T query(int l, int r){
        return query(r) - query(l -
1);
    }

    void update(int i, T val){
        for(; i <= n; i += i & -i)t[i]
+= val;
    }

    void update(int l, int r, T

```

```

val){
    update(l, val);
    update(r + 1, -val);
}
};

*** Seg tree Lazy***
template<typename T = int>
struct ST{
    #define lc node << 1
    #define rc node << 1 | 1

    int n;
    vector<int> v;
    vector<T> t, lazy;
    ST(vector<int> v) : v(v){
        n = v.size() - 1;
        t.assign(4 * n, 0);
        lazy.assign(4 * n, 0);
        build(1, 1, n);
    }

    void build(int node, int lo,
int hi){
        if(lo == hi){
            t[node] = v[lo];
            return;
        }
        int mid = (lo + hi) >> 1;
        build(lc, lo, mid);
        build(rc, mid + 1, hi);
        t[node] = pull(t[lc], t[rc]);
    }
    inline T pull(T a, T b){
//pull
        return a + b;
    }
    inline void push(int node, int
lo, int hi){ //lazy
        if(lazy[node] == 0)return;

        int val = lazy[node];
        t[node] += (hi - lo + 1) *
val;
        lazy[node] = 0;

        if(lo == hi)return;
        lazy[lc] = lazy[lc] + val;
        lazy[rc] = lazy[rc] + val;
    }

    void update(int node, int lo,
int hi, int l, int r, int val){
        push(node, lo, hi);
        if(r < lo or l > hi)return;
        if(lo >= l and hi <= r){
            lazy[node] += val; //update

```

```

    push(node, lo, hi);
    return;
}
int mid = (lo + hi) >> 1;
update(lc, lo, mid, l, r,
val);
update(rc, mid + 1, hi, l, r,
val);
t[node] = pull(t[lc], t[rc]);
}
T query(int node, int lo, int
hi, int l, int r){
    push(node, lo, hi);
    if(r < lo or l > hi) return 0;
    //return
    if(lo >= l and hi <= r) return
t[node];
    int mid = (lo + hi) >> 1;
    T q1 = query(lc, lo, mid, l,
r);
    T q2 = query(rc, mid + 1, hi,
l, r);
    return pull(q1, q2);
}
void update(int l, int r, int
val){
    update(1, 1, n, l, r, val);
}
T query(int l, int r){
    return query(1, 1, n, l, r);
}
};
***Iterative Segment Tree**
template<typename T = int>
struct ST{
    int n;
    vector<T> v, t;
    ST(vector<T> v) : v(v){
        n = v.size() - 1; //1-based
index
        t.resize(2 * n + 2);
        build();
    }
    const int R = 0;
    inline int Pull(int a, int b){
        return a + b;
    }
    void build(){
        for(int i = 1; i <= n; i++) t[i
+ n] = v[i];
        for(int i = n; i >= 1; i--){
            t[i] = pull(t[i << 1], t[i
<< 1 | 1]);
        }
    }
    void update(int i, T val){

```

```

        i += n;
        t[i] = val; //replace
        for(i >>= 1; i >= 1; i >>= 1){
            t[i] = pull(t[i << 1], t[i
<< 1 | 1]);
        }
    }
    T query(int l, int r){
        l += n; r += n;
        T ans = R;
        for(; l <= r; l >>= 1, r >>=
1){
            if(l % 2 == 1) ans =
pull(ans, t[l++]);
            if(r % 2 == 0) ans =
pull(ans, t[r--]);
        }
        return ans;
    }
};
***Merge Sort Tree***
struct MST{
    using vec = vector<int>;
    int n; vec v;
    vector<vec> t;
    MST(vec v) : v(v){
        n = v.size() - 1; //1-based
index
        t.resize(2 * n + 2);
        build();
    }
    inline void merge(vec &ans,
vec &a, vec &b) {
        int i = 0, j = 0;
        while(i < a.size() && j <
b.size()) {
            if(a[i] < b[j])
                ans.push_back(a[i++]);
            else ans.push_back(b[j++]);
        }
        while(i < a.size())
            ans.push_back(a[i++]);
        while(j < b.size())
            ans.push_back(b[j++]);
    }
    void build(){
        for(int i = 1; i <= n; i++) t[i
+ n].push_back(v[i]);
        for(int i = n; i >= 1; i--){
            merge(t[i], t[i << 1], t[i
<< 1 | 1]);
        }
    }
};

```

```

    bool flg = false;
    int count(const vec &v, int
x){
        if(flg){
            return (int)(v.end() -
upper_bound(v.begin(), v.end(),
x));
        }else{
            return
(int)(lower_bound(v.begin(),
v.end(), x) - v.begin());
        }
    }

    int query(int l, int r, int x,
bool f){
        flg = f; //flg = 0: strictly
less than x ; flg = 1: strictly
greater than x;
        l += n; r += n;
        int ans = 0;
        for(; l <= r; l >>= 1, r >>=
1){
            if(l % 2 == 1) ans +=
count(t[l++], x);
            if(r % 2 == 0) ans +=
count(t[r--], x);
        }
        return ans;
    }
};

// *** Sparse table ***
int t[N][18], v[N];
void build(int n){
    for(int i = 1; i <= n;
i++) t[i][0] = v[i];
    for(int j = 1; (1 << j) <= n;
j++){
        for(int i = 1; i + (1 << j) -
1 <= n; i++){
            t[i][j] = min(t[i][j - 1],
t[i + (1 << (j - 1))][j - 1]);
        }
    }
}
int query(int l, int r){
    int j = 31 - __builtin_clz(r -
l + 1);
    return min(t[l][j], t[r - (1
<< j) + 1][j]);
}

***Trie**
struct Trie {
    static const int B = 31;

```

```

struct node {
    node *nxt[2];
    int sz;
    node() {
        memset(nxt, 0,
sizeof(nxt));
        sz = 0;
    }
} *root;
Trie() {
    root = new node();
}
void insert(int val) {
    node *cur = root;
    cur->sz++;
    for (int i = B - 1; i >= 0;
i--) {
        int b = val >> i & 1;
        if (cur->nxt[b] == NULL)
            cur->nxt[b] = new
node();
        cur = cur->nxt[b];
        cur->sz++;
    }
}
void remove(int x) {
    node *cur = root;
    cur->sz--;
    for(int i = B - 1 ; i >= 0 ;
i--) {
        int b = x >> i & 1;
        node *nxt = cur->nxt[b];
        nxt->sz--;
        if(nxt->sz == 0) {
            // delete(nxt);
            // cur->nxt[b] = NULL;
            return;
        }
        cur = nxt;
    }
}
int query(int x, int k){ //
number of values s.t. val ^ x <
k
    node *cur = root;
    int ans = 0;
    for (int i = B - 1; i >= 0;
i--) {
        if (cur == NULL)
            break;
        int b1 = x >> i & 1, b2 =
k >> i & 1;
        if (b2 == 1) {
            if (cur->nxt[b1])
                ans +=
cur->nxt[b1]->sz;

```

```

        cur = cur->nxt[!b1];
    }
    else
        cur = cur->nxt[b1];
    }
    return ans;
}
int get_max(int x) { //
returns maximum of val ^ x
    node *cur = root;
    int ans = 0;
    for (int i = B - 1; i >= 0;
i--) {
        int k = x >> i & 1;
        if (cur->nxt[!k])
            cur = cur->nxt[!k], ans
<= 1, ans++;
        else
            cur = cur->nxt[k], ans
<= 1;
    }
    return ans;
}
int get_min(int x) { //
returns minimum of val ^ x
    node *cur = root;
    int ans = 0;
    for (int i = B - 1; i >= 0;
i--) {
        int k = x >> i & 1;
        if (cur->nxt[k])
            cur = cur->nxt[k], ans
<= 1;
        else
            cur = cur->nxt[!k], ans
<= 1, ans++;
    }
    return ans;
}
void del(node *cur) {
    for (int i = 0; i < 2; i++)
        if (cur->nxt[i])
            del(cur->nxt[i]);
    delete (cur);
}
}
// Trie t;
// t.insert(5);
// t.insert(10);
// int ans1 = t.query(3, 7);
// Count numbers where (val ^ 3)
< 7
//     cin x. cur ^= x;
t.query(cur, k). t.insert(cur);
//     for this, don't use
t.insert(0)

```

```

// int ans2 = t.get_max(3); //
Max (val ^ 3)
// int ans3 = t.get_min(3); //
Min (val ^ 3)

**SQRT**
const int N = 2e5 + 5;
const int sqN = 450;
long long a[N];
long long b[sqN];
int n, len; // n , len

void build() {
    for(int i = 0 ; i < n ; i++) {
        b[i / len] += a[i];
    }
}
int query(int l, int r) {
    int res = 0;
    while(l <= r) {
        if((l % len == 0) and ((l +
len - 1) <= r)) {
            res += b[l / len];
            l += len;
        }
        else {
            res += a[l++];
        }
    }
    return res;
}
// SQRT impl. basis [l, r] range
sum
// TC -> build: O(n)
// -> query: O(sqrtN)

**MO's Algorithm**
const int BS = 400;
int n, m, v[N];
struct Q{int id, l, r; };
bool cmp(Q &a, Q &b){
    if(a.l / BS == b.l / BS) return
a.r > b.r;
    return a.l / BS < b.l / BS;
}

int add(int i){
    return v[i];
}
int rem(int i){
    return v[i];
}

void Solve(){
    cin >> n >> m;

```

```

    for(int i = 1; i <= n; i++)cin
>> v[i];
    vector<Q> query(m);
    for(int i = 0; i < m; i++){cin
>> query[i].l >> query[i].r;
    query[i].id = i;}
    sort(query.begin(),
    query.end(), cmp);

vector<int> ans(m);
    int l = 0, r = 0, res = 0;
    for(auto q : query){
        while(r < q.r)res +=
add(++r);
        while(r > q.r)res -=
rem(r--);
        while(l > q.l)res +=
add(--l);
        while(l < q.l)res -=
rem(l++);
        ans[q.id] = res;
    }
    for(int i = 0; i < m; i++)cout
<< ans[i] << endl;
}

```

ORDERED_SET

```

#include<ext/pb_ds/assoc_contain
er.hpp>
#include<ext/pb_ds/tree_policy.h
pp>
using namespace __gnu_pbds;
template <class T>
using orderedSet =
    tree<T, null_type, less<T>,
rb_tree_tag,
tree_order_statistics_node_updat
e>;

```

DSU

```

struct Dsu{
    vector<int> par, rank, sz;
    Dsu(int n = 2e5): par(n + 1),
rank(n + 1), sz(n + 1, 1){
        iota(par.begin(), par.end(),
0);
    }
    int Find(int v){
        return par[v] = (par[v] == v)
? v : Find(par[v]);
    }
    void Union(int a, int b){
        a = Find(a);
        b = Find(b);
        if(a == b)return;
        if(rank[a] < rank[b])swap(a,

```

```

b);
        if(rank[a] ==
rank[b])rank[a]++;
        par[b] = a; sz[a] += sz[b];
    }
};

** Dynamic Bitset***
struct Cool_Bitset {
    vector<uint64_t> bits;
    int64_t b, n;
    Cool_Bitset(int64_t _b = 0) {
        init(_b);
    }
    void init(int64_t _b) {
        b = _b;
        n = (b + 63) / 64;
        bits.resize(n, 0);
    }
    void clear() {
        b = n = 0;
        bits.clear();
    }
    void reset() {
        bits.resize(n, 0);
    }
    void _clean() {
        // Reset all bits after `b`.
        if (b != 64 * n)
            bits.back() &= (1LLU << (b
- 64 * (n - 1))) - 1;
    }
    bool get(int64_t index) const
{
        return bits[index / 64] >>
(index % 64) & 1;
    }
    void set(int64_t index, bool
value) {
        assert(0 <= index && index <
b);
        bits[index / 64] &= ~(1LLU
<< (index % 64));
        bits[index / 64] |=
uint64_t(value) << (index % 64);
    }
    // Simulates `bs |= bs <<
shift;`
    void left_shift(int64_t
shift) {
        int64_t div = shift / 64,
mod = shift % 64;
        if (mod == 0) {
            for (int64_t i = n - 1; i
>= div; i--)
                bits[i] |= bits[i - div];

```

```

        return;
    }
    for (int64_t i = n - 1; i >=
div + 1; i--)
        bits[i] |= bits[i - (div +
1)] >> (64 - mod) | bits[i -
div] << mod;

    if (div < n)
        bits[div] |= bits[0] <<
mod;

    _clean();
}
// Simulates `bs |= bs >>
shift;`
void right_shift(int64_t
shift) {
    int64_t div = shift / 64,
mod = shift % 64;

    if (mod == 0) {
        for (int64_t i = div; i <
n; i++)
            bits[i - div] |= bits[i];

        return;
    }

    for (int64_t i = 0; i < n -
(div + 1); i++)
        bits[i] |= bits[i + (div +
1)] << (64 - mod) | bits[i +
div] >> mod;

    if (div < n)
        bits[n - div - 1] |= bits[n
- 1] >> mod;

    _clean();
}

int64_t find_first() const {
    for (int i = 0; i < n; i++)
        if (bits[i] != 0)
            return 64 * i +
__builtin_ctzll(bits[i]);

    return -1;
}
int64_t find_next(int x)
const {
    for (int i = x + 1; i < x +
64; i++)
        {

```



```

    if (get(i))
        return i;
}
for (int i = x / 64 + 1; i <
n; i++)
    if (bits[i] != 0)
        return 64 * i +
__builtin_ctzll(bits[i]);

return -1;
}
Cool_Bitset& operator&=(const
Cool_Bitset &other) {
    assert(b == other.b);

    for (int i = 0; i < n; i++)
        bits[i] &= other.bits[i];

    return *this;
}
};

```

*** Hashing Interval ***

```

mt19937
rng(chrono::steady_clock::now().
time_since_epoch().count());
#define rng(x,y)
uniform_int_distribution<int>(x,
y)(rng)
#define ll long long
typedef uint64_t ull;
struct H {
    ull x; H(ull _x=0) : x(_x) {}
    H operator+(H o) { return x +
o.x + (x + o.x < x); }
    H operator-(H o) { return
*this + ~o.x; }
    H operator*(H o) { auto m =
(__uint128_t)x * o.x;
return H((ull)m) + (ull)(m >>
64); }
    ull get() const { return x +
!~x; }
    bool operator==(H o) const {
return get() == o.get(); }
    bool operator<(H o) const {
return get() < o.get(); }
    friend std::ostream& operator
<< (std::ostream &os, const H
&h){ return os << h.x; }
    friend std::istream& operator
>> (std::istream &is, H &h){int
x; cin >> x; return h.x = x,
is;}

```

```

};
static const H C =
(1ll)rng(1e10,1e12); // (order ~
3e9; random also ok)

struct Hash {
    vector<H> ha, pw;
    Hash(string& str) :
ha(str.size() + 1), pw(ha) {
        pw[0] = 1;
        for (int i = 0; i <
str.size(); i++)
            ha[i+1] = ha[i] * C +
str[i],
            pw[i+1] = pw[i] * C;
    }
    H range(int a, int b) { //
hash [a, b)
    return ha[b] - ha[a] * pw[b -
a];
}
    void insert(char a){
        ha.push_back(ha.back()*C + a);
        pw.push_back(pw.back()*C);
    }
    /*
    bool isPalindrom(int l, int
r){
        int i = n - 1 - r;
        int j = n - 1 - l;
        return st.range(l, r + 1).x
== rv.range(i, j + 1).x;
    }
    */
};
*** Hashing with update***
mt19937
rng(chrono::steady_clock::now().
time_since_epoch().count());
#define rng(x,y)
uniform_int_distribution<int>(x,
y)(rng)
#define H int
const H C = rng(8.8e8, 1e9);

struct HashST{
    int n; string str;
    vector<H> t, pw, inv;
    HashST(string s){
        str = "#" + s;
        n = str.size() - 1;
        pw.resize(n + 2);
        inv.resize(n + 2);
        t.resize(2 * n + 2);
    }

```

```

        build();
    }
    void build(){
        pw[0] = pw[1] = 1;
        inv[0] = inv[1] = 1;
        H iC = mmiOfX(C);
        //multiplicative inverse
        for(int i = 1; i <= n; i++){
            t[i + n] = pw[i] * str[i]
% M;
            pw[i + 1] = pw[i] * C % M;
            inv[i + 1] = inv[i] * iC %
M;
        }
        for(int i = n; i >= 1; i--){
            t[i] = (t[i << 1] + t[i <<
1 | 1]) % M;
        }
    }
    void update(int i, char c){
        str[i] = c; i += n;
        t[i] = pw[i - n] * c % M;
        for(i >= 1; i; i >= 1){
            t[i] = (t[i << 1] + t[i <<
1 | 1]) % M;
        }
    }
    H query(int l, int r){
        H div = inv[1];
        l += n; r += n;
        H ans = 0;
        for(; l <= r; l >= 1, r >=
1){
            if(l % 2 == 1)ans = (ans +
t[l++] % M;
            if(r % 2 == 0)ans = (ans +
t[r--] % M;
        }
        return ans * div % M;
    }
};
***Z Algorithm***
vector<int> z_algo(string s){
    int n = s.size();
    vector<int> z(n);
    for(int i = 1, l = 0, r = 0; i
< n; i++){
        if(i <= r) z[i] = min(r - i
+ 1, z[i - l]);
        while(i + z[i] < n &&
s[z[i]] == s[i + z[i]]) ++z[i];
        if(i + z[i] - 1 > r) l = i,
r = i + z[i] - 1;
    }
    return z;
}

```

***KMP**

```
vector<int> Lps(string &str){
    int m = str.size();
    vector<int> lps(m);
    for(int i = 1, j = 0; i < m;){
        if(str[i] == str[j]){
            lps[i] = j + 1;
            i++; j++;
        }else{
            if(j)j = lps[j - 1];
            else lps[i] = 0, i++;
        }
    }
    return lps;
}

bool kmp(string str, string p){
    vector<int> lps = Lps(p);
    int n = str.size(), m = p.size();

    for(int i = 0, j = 0; i < n;
){
        if(str[i] == p[j]){
            i++; j++;
        }else{
            if(j)j = lps[j - 1];
            else i++;
        }
        if(j == m)return true;
    }
    return false;
}
```

Manachar

```
string preprocess(string &str){
    string res = "*";
    for(auto &i : str){
        res.push_back('#');
        res.push_back(i);
    }
    res += "#?";
    return res;
}

string LPS(string &inp){
    string str = preprocess(inp);
    int n = str.size(), c = 0, r = 0, mx = 0;
    vector<int> lps(n, 0);

    for(int i = 1; i < n - 1; i++){
        int mirror_i = 2 * c - i;
        if(i < r) lps[i] = min(r - i, lps[mirror_i]);
        while(str[i + lps[i] + 1] == str[i - lps[i] - 1]) lps[i]++;
    }
```

```
        if(i + lps[i] > r){
            c = i;
            r = i + lps[i];
        }
        if(lps[i] > lps[mx])mx = i;
    }
    return inp.substr((mx - lps[mx]) / 2, lps[mx]);
}

***Aho corasic**
struct node{
    node* nxt[26];
    node* suffix_link;
    node* output_link;
    int cnt[26];
    int ends;
    node(){
        for (int i = 0; i < 26; i++)
        {
            nxt[i] = nullptr;
            cnt[i] = 0;
        }
        suffix_link = nullptr;
        output_link = nullptr;
        ends = -1;
    }
};

struct AhoCorasic{
    node* root;
    char ch;
    int k;
    AhoCorasic(){
        root = new node();
        ch = 'a';
        k = 26;
    }
    void build_suffix_output_links(){
        root->suffix_link = root;
        queue<node*> q;
        for (int i = 0; i < k; i++)
        if (root->nxt[i]) {
            q.push(root->nxt[i]);
            root->nxt[i]->suffix_link = root;
        }
        while (!q.empty()) {
            node* cur = q.front();
            q.pop();
            for (int i = 0; i < k; i++)
            if (cur->nxt[i]) {
                node* tmp = cur->suffix_link;
                while ((!tmp->nxt[i]) && tmp != root) tmp = tmp->suffix_link;
            }
```

```
            if (tmp->nxt[i])
                cur->nxt[i]->suffix_link = tmp->nxt[i];
            else
                cur->nxt[i]->suffix_link = root;
            q.push(cur->nxt[i]);
        }
        if (cur->suffix_link->ends >= 0) cur->output_link = cur->suffix_link;
        else cur->output_link = cur->suffix_link->output_link;
    }

    void insert(string &words, int index){
        node* cur = root;
        for (int i = 0; i < words.size(); i++) {
            if (!cur->nxt[words[i] - ch]) {
                cur->nxt[words[i] - ch] = new node();
            }
            cur->cnt[words[i] - ch]++;
            cur = cur->nxt[words[i] - ch];
        }
        cur->ends = index;
    }

    void build(vector<string>&s){
        for (int i = 0; i < s.size(); i++)
            insert(s[i], i);
        build_suffix_output_links();
    }

    void search_end_indexes(string &text, vector<vector<int>>&arr){
        node* cur = root;
        int i = 0;
        while (i < text.size()) {
            if (cur->nxt[text[i] - ch]) {
                cur = cur->nxt[text[i] - ch];
                if (cur->ends >= 0)
                    arr[cur->ends].pb(i);
                node* mol = cur->output_link;
                while (mol) {
                    arr[mol->ends].pb(i);
                    mol = mol->output_link;
                }
            }
```

```

    }
    } else {
        while (cur != root &&
(!cur->nxt[text[i] - ch]))
            cur =
cur->suffix_link;
        if (cur->nxt[text[i] -
ch]) i--;
    }
    i++;
}
}
void dfs(node* cur){
    if (cur -> nxt[0]) dfs(cur
-> nxt[0]);
    if (cur -> nxt[1]) dfs(cur
-> nxt[1]);
}
void call_dfs(){
    dfs(root);
}
void selfDestruct(node* cur) {
    for(int i = 0; i < k; i++){
        if(cur->nxt[i])
selfDestruct(cur->nxt[i]);
    }
    delete cur;
}
~AhoCorasic() {
    selfDestruct(root);
}
};

***Suffix array nlogn***
template<char MIN_CHAR = '#',
int ALPHA = 256>
struct SuffixArray{
    int n;
    string s;
    vector<int> pos, sa, lcp;
    SuffixArray(){}
    SuffixArray(const string &s){
        s = _s + MIN_CHAR;
        n = s.size();
        pos.assign(n, 0);
        sa.assign(n, 0);
        lcp.assign(n - 1, 0);
        buildSA();
        buildLCP();
    }
    void buildSA(){
        vector<int> cnt(max(ALPHA,
(int)n), 0);
        for(int i = 0; i < n; i++)
cnt[s[i]]++;

```

```

        for(int i = 1; i < ALPHA;
i++) cnt[i] += cnt[i - 1];
        for(int i = n - 1; i >= 0;
i--) pos[--cnt[s[i]]] = i;

        for(int i = 1; i < n; i++){
            sa[pos[i]] = sa[pos[i -
1]] + (s[pos[i]] != s[pos[i -
1]]);
        }

        for(int k = 0; (1LL << k) <
n; k++){
            vector<int> npos(n),
nsa(n), ncnt(n, 0);

            for(int i = 0; i < n; i++)
pos[i] = (pos[i] - (1LL << k) +
n) % n;
            for(int i = 0; i < n; i++)
ncnt[sa[i]]++;
            for(int i = 1; i < n; i++)
ncnt[i] += ncnt[i - 1];
            for(int i = n - 1; i >= 0;
i--) npos[--ncnt[sa[pos[i]]]] =
pos[i];

            for(int i = 1; i < n;
i++){
                pair<int,int> cur =
{sa[npos[i]], sa[(npos[i] + (1LL
<< k)) % n]};
                pair<int,int> pre =
{sa[npos[i - 1]], sa[(npos[i -
1] + (1LL << k)) % n]};
                nsa[npos[i]] =
nsa[npos[i - 1]] + (cur != pre);
            }
            pos = npos; sa = nsa;
        }
    }

    void buildLCP(){
        for(int i = 0, k = 0; i < n
- 1; i++, k = max(k - 1, 0)){
            int j = pos[sa[i] - 1];
            while(s[i + k] == s[j +
k])k++;
            lcp[sa[i] - 1] = k;
        }
    }

    int count(string pat){
        int m = pat.size();
        int lo = 1, hi = n - 1, lb;

```

```

        while(lo < hi){
            int mid = (lo + hi) >> 1;
            if(s.compare(pos[mid], m,
pat) < 0) lo = mid + 1;
            else hi = mid;
        }
        if(s.compare(pos[lo], m,
pat) != 0)return 0;
        lb = lo;
        hi = n - 1;
        while(lo < hi){
            int mid = (lo + hi + 1) >>
1;
            if(s.compare(pos[mid], m,
pat) <= 0) lo = mid;
            else hi = mid - 1;
        }
        return lo - lb + 1;
    }

    int cntSubstr(){
        int len = n - 1;
        int ans = len * (len + 1) /
2;
        for(int x : lcp) ans -= x;
        return ans;
    }
};

*** Next greater left ***
stack<ll> stlmax;
vector<ll> lmax(n, -1);
for(i = n-1; i >= 0; i--) {
    while(!stlmax.empty() &&
a[stlmax.top()] < a[i]) {
        lmax[stlmax.top()] = i;
        stlmax.pop();
    }
    stlmax.push(i);
}

*** Next greater right ***
vector<int>
nextGreater(vector<int> &v){
    int n = v.size();
    vector<int> ans(n, -1);
    stack<int> st;
    for(int i = 0; i < n; i++){
        while(!st.empty() and v[i] >
v[st.top()]){
            ans[st.top()] = i;
            st.pop();
        }
        st.push(i);
    }
    return ans;
}

```

```

}
*** Next lower left ***
stack<ll> stlmin;
vector<ll> lmin(n, -1); //0
based indexing
for(i = n-1; i >= 0; i--) {
    while(!stlmin.empty() &&
a[stlmin.top()] > a[i]) {
        lmin[stlmin.top()] = i;
        stlmin.pop();
    }
    stlmin.push(i); //using index
}
*** Next lower right ***
stack<ll> strmin;
vector<ll> rmin(n, -1);
for(i = 0; i < n; i++) {
    while(!strmin.empty() &&
a[strmin.top()] > a[i]) {
        rmin[strmin.top()] = i;
        strmin.pop();
    }
    strmin.push(i);
}
*** Count inversion ***
ll mergeArray(vector<ll>& a, ll
low, ll mid, ll high) {
    vector<ll> left(a.begin() +
low, a.begin() + mid + 1);
    vector<ll> right(a.begin() +
mid + 1, a.begin() + high + 1);
    ll n = sz(left), m =
sz(right);
    ll i = 0, j = 0, k = low, cnt
= 0;
    while(i < n && j < m) {
        if(left[i] <= right[j]) {
            a[k++] = left[i++];
        } else {
            a[k++] = right[j++];
            cnt += (n - i);
        }
    }
    while(i < n) a[k++] =
left[i++];
    while(j < m) a[k++] =
right[j++];
    return cnt;
}
ll inversion(vector<ll>& a, ll
low, ll high) {
    ll inv = 0;
    if(low < high) {
        ll mid = (low + high) / 2;
        inv += inversion(a, low,

```

```

mid);
        inv += inversion(a, mid + 1,
high);
        inv += mergeArray(a, low,
mid, high);
    }
    return inv;
}
// *** Max element count in
range (l to r)
// for finding max occurred
element in range l to r
// it returns accurate value
when the count of maximum
element in range l to r is more
than n / 2, where n = r - l + 1
// example : 1 2 3 1 1, ans = 1
// example : 1 2 1 2, ans may be
1 or 2
// example 1 2 3 2, ans may be
1, 2 or 3
pll seg[4*N];
pll merge(pll x, pll y) {
    pll res;
    if(x.ff == y.ff) {
        res = {x.ff, x.ss + y.ss};
    } else {
        if(x.ss > y.ss) res = {x.ff,
x.ss - y.ss};
        else res = {y.ff, y.ss -
x.ss};
    }
    return res;
}
pll query(ll node, ll start, ll
end, ll l, ll r) {
    if(start >= l && end <= r)
return seg[node];
    if(start > r || end < l)
return {0, 0};
    ll mid = (start + end) >> 1;
    pll left = query(2 * node + 1,
start, mid, l, r);
    pll right = query(2 * node +
2, mid + 1, end, l, r);
    return merge(left, right);
}
void update(ll node, ll start,
ll end, ll ind, ll val) {
    if(start == end) {
        seg[node] = {val, 1};
        return;
    }
    ll mid = (start + end) >> 1;
    if(ind <= mid) update(2 * node
+ 1, start, mid, ind, val);

```

```

    else update(2 * node + 2, mid
+ 1, end, ind, val);
    seg[node] = merge(seg[2 * node
+ 1], seg[2 * node + 2]);
}

***Xor BASIS**
const int LG = 62;
struct Basis{
    int sz = 0; vector<int> basis
= vector<int>(LG + 1);
    void insert(int x){
        for(int i = LG; i >= 0;
i--){
            if(!(x & (1ll << i)))
continue;
            if(!basis[i]){
                basis[i] = x; sz++;
            }
            x ^= basis[i];
        }
    }
    bool can(int x){
        for(int i = LG; i >= 0;
i--){
            x = min(x, x ^ basis[i]);
        }
        return x == 0;
    }
    int getMax(int ans = 0){
        for(int i = LG; i >= 0;
i--){
            ans = max(ans, ans ^
basis[i]);
        }
        return ans;
    }
    void normalize(){
        for(int i = 0; i <= LG;
i++){
            if(!basis[i]) continue;
            for(int j = i - 1; j >= 0;
j--){
                if(basis[i] & (1ll <<
j)) basis[i] ^= basis[j];
            }
        }
        // 0-indexed kth smallest xor
value
        // call normalize() before
using
        int kthSmall(int k){

```

```

    if(k >= (1ll << sz)) return
-1;
    int ans = 0, id = 0;
    for(int i = 0; i <= LG;
i++){
        if(!basis[i]) continue;
        if(k & (1ll << id)) ans ^=
basis[i];
        id++;
    }
    return ans;
}
int kthLarge(int k){
    if(k >= (1ll << sz)) return
-1;
    return kthSmall((1ll << sz)
- 1 - k);
}
};

```

2D Geometry All in one

```

const double EPS = 1e-9;
const double PI = acos(-1.0);

int sign(double x){ return (x >
EPS) - (x < -EPS); }

struct point{
    double x, y;
    point(){ x = y = 0; }
    point(double _x, double _y){ x
= _x; y = _y; }
    point operator + (const point
&p) const { return point(x +
p.x, y + p.y); }
    point operator - (const point
&p) const { return point(x -
p.x, y - p.y); }
    point operator * (double k)
const { return point(x * k, y *
k); }
    point operator / (double k)
const { return point(x / k, y /
k); }
    bool operator == (const point
&p) const { return sign(x - p.x)
== 0 && sign(y - p.y) == 0; }
    bool operator < (const point
&p) const {
        if(sign(x - p.x) != 0)
return x < p.x;
        return y < p.y;
    }
};

```

```

double dot(point a, point b){
return a.x * b.x + a.y * b.y; }
double cross(point a, point b){
return a.x * b.y - a.y * b.x; }
double cross(point a, point b,
point c){ return cross(b - a, c
- a); }
double norm2(point a){ return
dot(a, a); }
double norm(point a){ return
sqrt(norm2(a)); }
double dist(point a, point b){
return norm(a - b); }
double angle(point a, point b){
return acos(max((double) -1.0,
min((double)1.0, dot(a, b) /
norm(a) / norm(b)))); }
double triangleArea(point a,
point b, point c){ return
fabs(cross(a, b, c)) / 2.0; }

```

```

point rotate(point a, double
rad){
    return point(a.x * cos(rad) -
a.y * sin(rad), a.x * sin(rad) +
a.y * cos(rad));
}

```

```

int orientation(point a, point
b, point c){ return
sign(cross(a, b, c)); }

```

```

bool onSegment(point a, point b,
point p){
    if(orientation(a, b, p) != 0)
return false;
    return p.x >= min(a.x, b.x) -
EPS && p.x <= max(a.x, b.x) +
EPS &&
        p.y >= min(a.y, b.y) -
EPS && p.y <= max(a.y, b.y) +
EPS;
}

```

```

bool segmentIntersect(point a,
point b, point c, point d){
    int o1 = orientation(a, b, c),
o2 = orientation(a, b, d);
    int o3 = orientation(c, d, a),
o4 = orientation(c, d, b);
    if(o1 != o2 && o3 != o4)
return true;
    if(o1 == 0 && onSegment(a, b,
c)) return true;
    if(o2 == 0 && onSegment(a, b,

```

```

d)) return true;
    if(o3 == 0 && onSegment(c, d,
a)) return true;
    if(o4 == 0 && onSegment(c, d,
b)) return true;
    return false;
}

```

```

bool areLinesParallel(point a,
point b, point c, point d){
    return sign(cross(b - a, d -
c)) == 0;
}

```

```

bool areLinesSame(point a, point
b, point c, point d){
    return areLinesParallel(a, b,
c, d) && sign(cross(b - a, c -
a)) == 0;
}

```

```

// line ab and line cd
intersection, false if parallel
bool lineIntersection(point a,
point b, point c, point d, point
&ans){
    point r = b - a, s = d - c;
    double den = cross(r, s);
    if(sign(den) == 0) return
false;
    ans = a + r * (cross(c - a, s)
/ den);
    return true;
}

```

```

point projectpointLine(point a,
point b, point p){
    point ab = b - a;
    if(sign(norm2(ab)) == 0)
return a;
    return a + ab * (dot(p - a,
ab) / norm2(ab));
}

```

```

point reflectionpointLine(point
a, point b, point p){
    point q = projectpointLine(a,
b, p);
    return q * 2 - p;
}

```

```

double distpointLine(point a,
point b, point p){
    if(a == b) return dist(a, p);
    return fabs(cross(b - a, p -

```

```

a)) / norm(b - a);
}

point projectpointSegment(point
a, point b, point p){
    point ab = b - a;
    if(sign(norm2(ab)) == 0)
return a;
    double t = dot(p - a, ab) /
norm2(ab);
    if(t < 0) return a;
    if(t > 1) return b;
    return a + ab * t;
}

double distpointSegment(point a,
point b, point p){
    return dist(p,
projectpointSegment(a, b, p));
}

double distSegmentSegment(point
a, point b, point c, point d){
    if(segmentIntersect(a, b, c,
d)) return 0;
    return
min({distpointSegment(a, b, c),
distpointSegment(a, b, d),

distpointSegment(c, d, a),
distpointSegment(c, d, b)});
}

double polygonArea(vector<point>
&p){
    double area = 0;
    int n = p.size();
    for(int i = 0; i < n; i++){
area += cross(p[i], p[(i + 1) %
n]);
    }
    return fabs(area) / 2.0;
}

double
polygonPerimeter(vector<point>
&p){
    double ans = 0;
    int n = p.size();
    for(int i = 0; i < n; i++) ans
+= dist(p[i], p[(i + 1) % n]);
    return ans;
}

bool
ispointOnPolygon(vector<point>

```

```

&p, point q){
    int n = p.size();
    for(int i = 0; i < n; i++){
        if(onSegment(p[i], p[(i + 1)
% n], q)) return true;
        return false;
    }

    // 0 = outside, 1 = inside, 2 =
boundary
    int pointInPolygon(vector<point>
&p, point q){
        if(ispointOnPolygon(p, q))
return 2;
        bool inside = false;
        int n = p.size();
        for(int i = 0, j = n - 1; i <
n; j = i++){
            point a = p[i], b = p[j];
            bool ok = ((a.y > q.y) !=
(b.y > q.y)) &&
                (q.x < (b.x - a.x) * (q.y
- a.y) / (b.y - a.y) + a.x);
            if(ok) inside = !inside;
        }
        return inside;
    }

    bool isConvex(vector<point> &p){
        int n = p.size(), pos = 0, neg
= 0;
        for(int i = 0; i < n; i++){
            int o = orientation(p[i],
p[(i + 1) % n], p[(i + 2) % n]);
            if(o > 0) pos = 1;
            if(o < 0) neg = 1;
        }
        return !(pos && neg);
    }

    // Monotonic chain Convex Hull
    // <= 0 removes collinear points
    // < 0 keeps collinear boundary
points
    vector<point>
convexHull(vector<point> p){
        sort(p.begin(), p.end());
        p.erase(unique(p.begin(),
p.end()), p.end());
        int n = p.size();
        if(n <= 1) return p;
        vector<point> h;
        for(int i = 0; i < n; i++){
            while(h.size() >= 2 &&
orientation(h[h.size() - 2],

```

```

h.back(), p[i]) <= 0)
h.pop_back();
            h.push_back(p[i]);
        }
        int lower = h.size();
        for(int i = n - 2; i >= 0;
i--){
            while((int)h.size() > lower
&& orientation(h[h.size() - 2],
h.back(), p[i]) <= 0)
h.pop_back();
            h.push_back(p[i]);
        }
        h.pop_back();
        return h;
    }

    // input must be convex hull in
CCW/CW order
    double
convexDiameter(vector<point>
&p){
        int n = p.size();
        if(n == 1) return 0;
        if(n == 2) return dist(p[0],
p[1]);
        double ans = 0;
        int j = 1;
        for(int i = 0; i < n; i++){
            int ni = (i + 1) % n;
            while(fabs(cross(p[ni] -
p[i], p[(j + 1) % n] - p[i])) >
                fabs(cross(p[ni] -
p[i], p[j] - p[i]))) j = (j + 1)
% n;
            ans = max(ans, dist(p[i],
p[j]));
        }
        return ans;
    }

    // 0 = outside, 1 = inside, 2 =
boundary
    // polygon must be strictly
convex and ordered
    int
pointInConvexPolygon(vector<point>
&p, point q){
        int n = p.size();
        if(n < 3) return
ispointOnPolygon(p, q) ? 2 : 0;
        if(onSegment(p[0], p[1], q) ||
onSegment(p[0], p[n - 1], q))
return 2;
        if(orientation(p[0], p[1], q)

```



```
< 0 || orientation(p[0], p[n - 1], q) > 0) return 0;
int l = 1, r = n - 1;
while(l + 1 < r){
    int mid = (l + r) >> 1;
    if(orientation(p[0], p[mid], q) >= 0) l = mid;
    else r = mid;
}
int o = orientation(p[l], p[(l + 1) % n], q);
if(o == 0) return 2;
return o > 0 ? 1 : 0;
}
```

// Integer Geometry / Pick's Theorem

```
struct Ipoint{
    int x, y;
    Ipoint(){}
    Ipoint(int _x, int _y){ x = _x; y = _y; }
};
```

```
int polygonArea2(vector<Ipoint> &p){
    int area = 0, n = p.size();
    for(int i = 0; i < n; i++){
        area += p[i].x * p[(i + 1) % n].y - p[i].y * p[(i + 1) % n].x;
    }
    return abs(area);
}
```

// excluding endpoints

```
int latticepointsOnSegment(Ipoint a, Ipoint b){
    return __gcd(abs(a.x - b.x), abs(a.y - b.y)) - 1;
}
```

// boundary lattice points of polygon

```
int boundarypoints(vector<Ipoint> &p){
    int ans = 0, n = p.size();
    for(int i = 0; i < n; i++){
        Ipoint a = p[i], b = p[(i + 1) % n];
        ans += __gcd(abs(a.x - b.x), abs(a.y - b.y));
    }
}
```

```
return ans;
}

// Pick's theorem: A = I + B / 2 - 1
// returns number of interior lattice points
int interiorpoints(vector<Ipoint> &p){
    int A2 = polygonArea2(p), B = boundarypoints(p);
    return (A2 - B + 2) / 2;
}
```

<*****Grandy Number*****>

The game starts with a number- 'n' and the player to move divides the number- 'n' with 2, 3 or 6 and then takes the floor. If the integer becomes 0, it is removed. The last player to move wins.

```
int calculateMex(unordered_set<int> Set)
{
    int Mex = 0;

    while (Set.find(Mex) != Set.end())
        Mex++;

    return (Mex);
}
```

```
int calculateGrundy (int n)
{
    if (n == 0)
        return (0);
    unordered_set<int> Set;
    Set.insert(calculateGrundy(n/2));
    Set.insert(calculateGrundy(n/3));
    Set.insert(calculateGrundy(n/6));
    return (calculateMex(Set));
}
```

bitmask formula

```
1. a + b = (a ^ b) + 2 * (a & b);
//here a , b is any integer number.
2. a + b gives the same parity as a ^ b
#define setbits(x) __builtin_popcountll(x)
#define zrbits(x) __builtin_ctzll(x)
#define unset_bit(x, inx) x = (x & ~(1ll << (inx)))
#define set_bit(x, idx) x = x | 1ll << idx
#define check_bit(x, idx) min(x & (1ll << idx), 1ll)
#define toggle_bit(x, idx) x = x ^ (1ll << idx)
```

Unordered Hashmap

```
struct custom_hash {
    static uint64_t
    splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) *
        0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) *
        0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x)
    const {
        static const uint64_t
        FIXED_RANDOM =
        chrono::steady_clock::now().time_s
        ince_epoch().count();
        return splitmix64(x +
        FIXED_RANDOM);
    }
};
```

<*****128BitIO*****>

```
__int128 read() {
    __int128 x = 0, f = 1;
    char ch = getchar();
    while (ch < '0' || ch > '9') {
        if (ch == '-') f = -1;
        ch = getchar();
    }
    while (ch >= '0' && ch <= '9') {
        x = x * 10 + ch - '0';
        ch = getchar();
    }
    return x * f;
}

void printi(__int128 x) {
    if (x < 0) {
        putchar('-');
        x = -x;
    }
    if (x > 9) printi(x / 10);
    cout << char(x % 10 + '0');
}
}
```

<***** PBDS *****>

```
// 1. delete a given number from the list
os.erase(5);

// when you use less equal
erase(ms.find_by_order(ms.order_of_key(
value)))

// 2. find a given number(7)
if (os.find(7) == os.end())
    cout << "not found\n";
else
    auto idx = os.find(7);
```

University of Rajshahi

// 3. how many numbers are smaller than a given value(7)

```
cout << os.order_of_key(7);
```

// 4. delete the k'th smallest number

```
os.erase(os.find_by_order(k));
```

// 5. delete the smallest number from the list

```
os.erase(os.begin());
```

// 6. delete the greatest number from the list

```
os.erase(os.rbegin());
```

// 7. what is the smallest number which is greater than or equal to a given number(7)

```
cout << *os.lower_bound(7) << "\n";
```

// 8. what is the smallest number which is greater than to a given number(7)

```
cout << *os.upper_bound(7) << "\n";
```

<*****Derangement*****>

```
int derangement(int n){
    if(n < 2) return 0;
    vector<int> dp(n + 1);
    dp[2] = 1;
    for(int i = 3; i <= n; i++){
        dp[i] = (i - 1) * (dp[i - 1] + dp[i - 2]) % M;
    }
    return dp[n];
}
```

// Grandy numbers

The number of ways to put identical objects into labelled boxes is

$$\binom{n+k-1}{n}.$$

**** **stirling number first kind = box! * stirling number second kind(not 100% sure)**

//Game Theory grandy number = mex{ for all reachables x from this state, grandy(x) }

// Pick's Theorem

Area of a polygon, $A = i + (b / 2) - 1$

Where, i = number of points inside the polygon, b = number of points on the boundary

//Catalan number

N-th term = $2nCn / (n + 1) = 2nCn - 2nC(n+1)$

1, 2, 5, 14, 42 ...

#python demo code

```
def solve(ct):
    a, b = map(int, input().split())
    tmp = a
    mm = {}
    nn = {}
    for i in range(2, int(a**0.5) + 1):
```

```
while a % i == 0:
```

```
    a //= i
```

```
    if i in mm:
```

```
        mm[i] += 1
```

```
    else:
```

```
        mm[i] = 1
```

```
if a > 1:
```

```
    mm[a] = 1
```

```
nn = mm.copy()
```

```
for key, value in mm.items():
```

```
    mm[key] = value * b
```

```
ans = []
```

```
prod = 1
```

```
for value in mm.values():
```

```
    prod *= (value + 1)
```

```
for key, value in mm.items():
```

```
    sum_value = (value * (value + 1)) // 2
```

```
    baki = prod // (value + 1)
```

```
    sum_value *= baki
```

```
    ans.append(sum_value // nn[key])
```

```
ans.sort()
```

```
print(ans[0] % mod)
```

```
def main():
```

```
    t = 1
```

```
    # t = int(input())
```

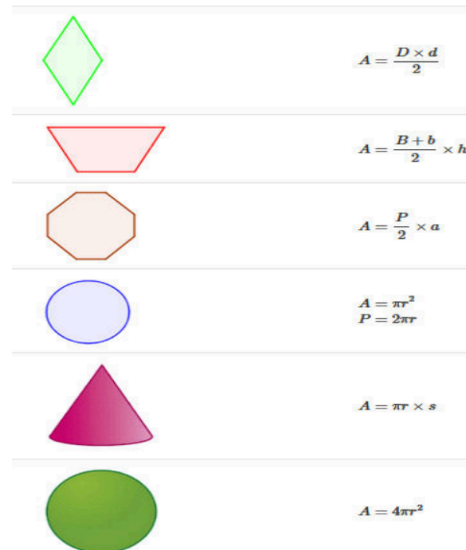
```
    for ct in range(1, t+1):
```

```
        solve(ct)
```

```
if __name__ == "__main__":
```

```
    mod = 998244353
```

```
    main()
```



$$\sum_{k=1}^n k = \frac{n(n+1)}{2}$$

$$\sum_{k=1}^n k^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{k=1}^n k^3 = \frac{n^2(n+1)^2}{4}$$

$$\sum_{k=1}^n k(k+1) = \frac{n(n+1)(n+2)}{3}$$

$$\sum_{k=1}^n \frac{1}{k(k+1)} = \frac{n}{n+1}$$

$$\sum_{k=1}^n k(k+1)(k+2) = \frac{n(n+1)(n+2)(n+3)}{4}$$

$$\sum_{k=1}^n \frac{1}{k(k+1)(k+2)} = \frac{n(n+3)}{4(n+1)(n+2)}$$

$$\sum_{k=1}^n (2k-1) = n^2$$

© easycalculation.com

Sum of n odd numbers = n^2

Sum of n even numbers = $n(n+1)$

Sum of geometric series

$$S_n = \frac{a(1 - r^n)}{(1 - r)}$$

//assign $x[n]=x[0]$ and $y[n]=y[0]$ and increase n to n+1 before calling..

```
long double polygonArea(int x[],int y[],int n)
{
    int area = 0;
    for(int i=0;i<n;i++){
        area += x[i]*y[i+1]-x[i+1]*y[i];
    }
    long double ar = area;
    ar/=2;
    return ar;
}
```

- **Area of a Circle** = $A = \pi r^2$
- **Circumference of a Circle** = $2\pi r$
- The curved **surface area of a Cylinder** = $2\pi rh$
- Total surface area of a Cylinder = $2\pi r(r + h)$
- **Volume of a Cylinder** = $V = \pi r^2 h$
- The curved **surface area of a cone** = $\pi r l$
- Total surface area of a cone = $\pi r(r+l) = \pi r[r + \sqrt{(h^2 + r^2)}]$
- **Volume of a Cone** = $V = \frac{1}{3} \times \pi r^2 h$
- **Surface Area of a Sphere** = $S = 4\pi r^2$
- **Volume of a Sphere** = $V = \frac{4}{3} \times \pi r^3$

 $V = s^3$

 $V = l \times w \times h$

 $V = b \times h$

 $V = \pi r^2 \times h$

 $V = \frac{1}{3} b \times h$

 $V = \frac{4}{3} \pi r^3$

4.Parallelogram	Perimeter, $P = 2(a + b)$
	Area, $A = bh$
	Height, $h = A/b$
	Base, $b = A/h$
	Where, a and b are the sides of a parallelogram h = height of a parallelogram
5. Trapezium	Area, $A = \frac{1}{2}(a + b)h$
	Height, $h = 2A/(a + b)$
	Base, $b = 2(A/h) - a$
	Where, a and b are the parallel sides h = distance between two parallel sides

```
****Egg Dropping DP***
int minTrials(int n, int k)
{
    // Initialize array of size (n+1) and m as moves.
    int dp[n + 1] = { 0 }, m;
    for (m = 0; dp[n] < k; m++) {
        for (int x = n; x > 0; x--) {
            dp[x] += 1 + dp[x - 1];
        }
    }
    return m;
}
/* Driver code*/
int main()
{
    int n = 2, k = 36;
    cout<<"Minimum number of trials "
```

```
"in worst case with "
<< n << " eggs and " << k << " floors is
";
cout << minTrials(2, 36);
return 0;
}
// *** floyd warshal ***
vector<vector<ll>> dist(N, vector<ll>(N));
ll n, m, q;
void floyd_warshaal() {
    for(ll k = 1; k <= n; k++) {
        for(ll i = 1; i <= n; i++) {
            for(ll j = 1; j <= n; j++) {
                dist[i][j] = min(dist[i][j], dist[i][k] +
dist[k][j]);
            }
        }
    }
}
void calculate() {
    for(ll i = 1; i <= n; i++) {
        for(ll j = 1; j <= n; j++) {
            if(i == j) dist[i][j] = 0; // need to be
edited
            else dist[i][j] = INF;
        }
    }
    for(ll i = 1; i <= m; i++) {
        ll x, y, w;
        cin >> x >> y >> w;
        dist[x][y] = min(dist[x][y], w);
        dist[y][x] = min(dist[y][x], w);
    }
    floyd_warshaal();
}
*** FFT multiply ***
using cd = complex<double>;
const double PI = acos(-1);
void fft(vector<cd> &p, bool flg){
    int n = p.size();
    if(n == 1)return;
    vector<cd> pe(n / 2), po(n / 2);
    for(int i = 0; i < n / 2; i++){
        pe[i] = p[2 * i];
        po[i] = p[2 * i + 1];
    }
    fft(pe, flg);
    fft(po, flg);
    double ang = (flg ? -1 : 1) * 2
* PI / n;
```

```
cd w(1), wd(cos(ang), sin(ang));

for(int i = 0; i * 2 < n; i++){
    p[i] = pe[i] + w * po[i];
    p[i + n / 2] = pe[i] - w *
po[i];
    if(fl){
        p[i] /= 2;
        p[i + n / 2] /= 2;
    }
    w *= wd;
}
}
vector<int> mult(vector<int> &a,
vector<int> &b){
    vector<cd> fa(a.begin(),
a.end()), fb(b.begin(), b.end());
    int n = 1;
    while(n < a.size() + b.size())n
<= 1;
    fa.resize(n);
    fb.resize(n);
    fft(fa, false);
    fft(fb, false);
    for(int i = 0; i < n; i++)fa[i]
*= fb[i];
    fft(fa, true);
    vector<int> res(n);
    for(int i = 0; i < n; i++)res[i]
= round(fa[i].real());
    return res;
}

// Custom set
struct cmp {
    bool operator() (int a, int b)
const {
        return a < b;
    }
};
// Custom map
struct mydata{
    int a, b;
};
struct cmp{
    bool operator() (const mydata&
lhs, const mydata& rhs) const
    {
        return lhs.a > rhs.a;
    }
};
map<mydata, int, cmp> mp;
// Bitset functions
```

Function Name	Function Description
<code>set()</code>	Set the bit value at the given index to 1.
<code>reset()</code>	Set the bit value at a given index to 0.
<code>flip()</code>	Flip the bit value at the given index.
<code>count()</code>	Count the number of set bits .
<code>test()</code>	Returns the boolean value at the given index.
<code>any()</code>	Checks if any bit is set.
<code>none()</code>	Checks if none bit is set.
<code>all()</code>	Check if all bit is set.
<code>size()</code>	Returns the size of the bitset .
<code>to_string()</code>	Converts bitset to std::string .
<code>to_ulong()</code>	Converts bitset to unsigned long.
<code>to_ullong()</code>	Converts bitset to unsigned long long.

Bit functions

1. __builtin_popcount(x)

This function is used to count the number of one's(set bits) in an integer.

2 . __builtin_parity(x)

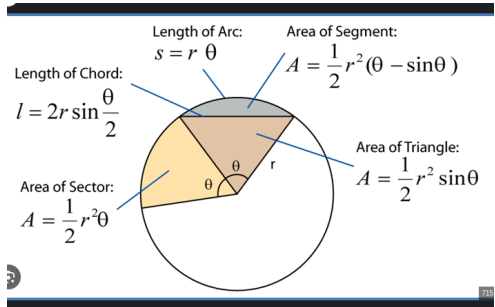
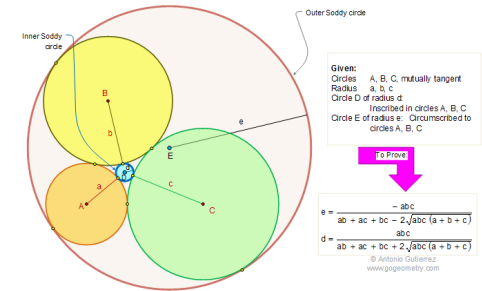
This function is used to check the parity of a number. This function returns true(1) if the number has odd parity else it returns false(0) for even parity.

3.__builtin_clz(x)

This function is used to count the leading zeros of the integer.

4.__builtin_ctz(x)

This function is used to count the trailing zeros of the given



3.If 3 medians given then, the area of the triangle will be

Area = 4/3*sqrt(s*(s-a)*(s-b)*(s-c)))

// Random test case generator

freopen("input.txt", "w", stdout);

mt19937

```
rng(chrono::steady_clock::now().time_since_ep
och().count());
#define rng(x,y)
uniform_int_distribution<int>(x,y)(rng)
```

```
void print_array_randomly(vector<int>a){
    while(a.size()) {
        int x = rng(0, a.size()-1);
        cout << a[x] << " ";
        a.erase(a.begin()+x);
    }
    cout << endl;
}
```

// Lamda function

```
function<void(int,int)> x = [&]
(int x, int y){
    return 1;
};
```

Combinatorics

General

- 1. $\sum_{0 \leq k \leq n} \binom{n-k}{k} = Fib_{n+1}$
- 2. $\binom{n}{k} = \binom{n}{n-k}$
- 3. $\binom{n}{k} + \binom{n}{k+1} = \binom{n+1}{k+1}$
- 4. $k \binom{n}{k} = n \binom{n-1}{k-1}$
- 5. $\binom{n}{k} = \frac{n}{k} \binom{n-1}{k-1}$
- 6. $\sum_{i=0}^n \binom{n}{i} = 2^n$
- 7. $\sum_{i \geq 0} \binom{n}{2i} = 2^{n-1}$
- 8. $\sum_{i \geq 0} \binom{n}{2i+1} = 2^{n-1}$
- 9. $\sum_{i=0}^k (-1)^i \binom{n}{i} = (-1)^k \binom{n-1}{k}$
- 10. $\sum_{i=0}^k \binom{n+i}{i} = \sum_{i=0}^k \binom{n+i}{n} = \binom{n+k+1}{k}$
- 11. $1 \binom{n}{1} + 2 \binom{n}{2} + 3 \binom{n}{3} + \dots + n \binom{n}{n} = n2^{n-1}$
- 12. $1^2 \binom{n}{1} + 2^2 \binom{n}{2} + 3^2 \binom{n}{3} + \dots + n^2 \binom{n}{n} = (n+n^2)2^{n-2}$

i=r \

- 15. $\sum_{i=0}^k \binom{k}{i}^2 = \binom{2k}{k}$
- 16. $\sum_{k=0}^n \binom{n}{k} \binom{n}{n-k} = \binom{2n}{n}$
- 17. $\sum_{k=q}^n \binom{n}{k} \binom{k}{q} = 2^{n-q} \binom{n}{q}$
- 18. $\sum_{i=0}^n k^i \binom{n}{i} = (k+1)^n$
- 19. $\sum_{i=0}^n \binom{2n}{i} = 2^{2n-1} + \frac{1}{2} \binom{2n}{n}$
- 20. $\sum_{i=1}^n \binom{n}{i} \binom{n-1}{i-1} = \binom{2n-1}{n-1}$
- 21. $\sum_{i=0}^n \binom{2n}{i}^2 = \frac{1}{2} \left(\binom{4n}{2n} + \binom{2n}{n}^2 \right)$

Fibonacci Number

85. $F_0 = 0, F_1 = 1$ and $F_n = F_{n-1} + F_{n-2}$

86. $F_n = \sum_{k=0}^{\lfloor \frac{n-1}{2} \rfloor} \binom{n-k-1}{k}$

87. $F_n = \frac{1}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2} \right)^n - \frac{1}{\sqrt{5}} \left(\frac{1-\sqrt{5}}{2} \right)^n$

88. $\sum_{i=1}^n F_i = F_{n+2} - 1$

89. $\sum_{i=0}^n F_{2i+1} = F_{2n}$

90. $\sum_{i=1}^n F_{2i} = F_{2n+1} - 1$

91. $\sum_{i=1}^n F_i^2 = F_n F_{n+1}$

92. $F_m F_{n+1} - F_{m-1} F_n = (-1)^n F_{m-n}$

$F_{2n} = F_{n+1}^2 - F_{n-1}^2 = F_n (F_{n+1} + F_{n-1})$

93. $F_m F_n + F_{m-1} F_{n-1} = F_{m+n-1}$

$F_m F_{n+1} + F_{m-1} F_n = F_{m+n}$

★ Pair forming LCM

ans = 0
i = 1...n
j = i...n
if (LCM(i, j) == n)
ans++;
equation: n/2 * (1 + (2*e1 + 1) * (2*e2 + 1) *...)

★ GCD Extreme

ans = 0
i = 1...N
j = i+1 ... N
ans += gcd(i, j)
equation: sum of d * phi(N/d)

★ LCM Extreme

ans = 0
i = 1...n
j = i+1 ... n
ans += lcm(i, j);
equation: n/2 * (1 + sum of (phi(d) * d) of all)
d means divisors of n

★ LCM Sum

ans = 0
i = 1...N
ans += lcm(i, N)
equation: n/2 * (1 + sum of (phi(d) * d) of all)

GCD and LCM

- 174. $\gcd(\text{lcm}(a, b), \text{lcm}(b, c), \text{lcm}(a, c)) = \text{lcm}(\gcd(a, b), \gcd(b, c), \gcd(a, c))$
- 175. $\gcd(A_L, A_{L+1}, \dots, A_R) = \gcd(A_L, A_{L+1} - A_L, \dots, A_R - A_{R-1})$
- 176. Given n, if $SUM = LCM(1, n) + LCM(2, n) + \dots + LCM(n, n)$
then $SUM = \frac{n}{2} \left(\sum_{d|n} (\phi(d) \times d) + 1 \right)$

== Stress Testing

generator file::

```
mt19937_64
rng(chrono::steady_clock::now().time_since_ep
och().count());
inline int gen_random(int l, int r) {
    return uniform_int_distribution<int>(l, r)(rng);
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);
```

```
freopen("input.in", "w", stdout);
```

```
}
```

```
checker file::
```

```
g++ -std=gnu++17 -O2 gen.cpp -o gen
```

```
for ((i = 1; i <= 500; i++)); do
```

```
    echo "Test $i"
```

```
    ./gen > input.txt
```

```
    if diff -q <(/optimal < input.txt) <(/brute <
```

```
input.txt) > /dev/null; then
```

```
        continue
```

```
    else
```

```
        echo "Mismatch found!"
```

```
        echo "Saving failing test to fail.in"
```

```
        cp input.txt fail.in
```

```
        echo "Input:"
```

```
        cat input.txt
```

```
        echo "Optimal:"
```

```
        ./optimal < input.txt
```

```
        echo "Brute:"
```

```
        ./brute < input.txt
```

```
        exit 0
```

```
    fi
```

```
done
```